

Hochschule Pforzheim
Fakultät für Technik
Studiengang Künstliche Intelligenz

Externes Parkassistenzsystem

Projektarbeit

Erstprüfer Dipl.-SpOec. Annegret Zimmermann

vorgelegt von **Serhat Subasi, Tom Liebegall, Daniel
Egbe, Vincent Gentz**

Matrikelnummer 327488, 332349, 333106, 331843

Abgabetermin 19.12.2025

Eigenständigkeitserklärung Hochschule Pforzheim

Bei der Abgabe einer schriftlichen Arbeit haben die Studierenden schriftlich zu versichern, dass sie ihre Arbeit - bei einer Gruppenarbeit ihren entsprechend gekennzeichneten Teil der Arbeit - selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben. Studierende können in Absprache mit der Erstprüferin bzw. dem Erstprüfer eine vom Muster abweichende Selbstständigkeitserklärung verwenden, sofern diese inhaltlich dem Muster entspricht. Folgend ein Muster für die Selbstständigkeitserklärung:

„Hiermit versichere wir Vincent Gentz, Tom Liebegall, Daniel Egbe und Serhat Subasi die vorliegende Projektarbeit selbständig verfasst zu haben. Die aus fremden Quellen direkt oder indirekt übernommenen Texte, Gedankengänge, Konzepte, Grafiken usw. habe ich als solche gekennzeichnet und mit vollständigen Verweisen auf die jeweilige Quelle versehen.

Weiter wurde die Arbeit bisher nicht an einer anderen Stelle zur Prüfung vorgelegt.

Außerdem versichere ich... (eine der Optionen ist in Absprache zwischen Prüfenden und Geprüften verbindlich auszuwählen und anzukreuzen):

☒ **Option 1: Erlaubnis KI-gestützter Assistenzwerkzeuge ohne Kennzeichnungspflicht (unterstützende Anwendung)**

Ich versichere, dass ich mich KI-gestützter Assistenzwerkzeuge lediglich unterstützend bedient habe (z.B. Rechtschreib- und Grammatikprüfung, Formatierung, Verbesserung von Übersetzungen oder eigener Formulierungen) und in der vorliegenden Arbeit mein gestalterischer Einfluss überwiegt. Ich bin mir bewusst, dass die Nutzung maschinell generierter Inhalte keine Garantie für Qualität und Korrektheit gewährleistet und verantworte die Übernahme jeglicher von mir verwendeter maschinell generierter Passagen vollumfänglich selbst. Ich versichere, dass ich keine KI-Werkzeuge verwendet habe, deren Nutzung der Prüfer/ die Prüferin explizit schriftlich ausgeschlossen hat oder die kennzeichnungspflichtig sind.

☐ **Option 2: Kennzeichnungspflicht KI-generierter Inhalte im Falle einer erlaubten Nutzung**

Ich habe Künstliche Intelligenz als Ausführungs- und Gestaltungswerkzeug eingesetzt, bin mir jedoch bewusst, dass die Nutzung maschinell generierter Texte, Grafiken, Bilder, Bewegtbilder, Codes etc. keine Garantie für Qualität und Korrektheit gewährleistet. Ich verantworte die Übernahme jeglicher von mir verwendeter

maschinell generierter Passagen, vollumfänglich selbst und trage Verantwortung, falls es zu fehlerhaften Inhalten, zu Verstößen gegen das Datenschutzrecht, das Urheberrecht oder zu wissenschaftlichem Fehlverhalten (z.B. Plagiate) kommt.

Ich versichere, dass in der vorliegenden Arbeit mein intellektueller und gestalterischer Einfluss überwiegt und ich beim Einsatz KI-gestützter Werkzeuge steuernd gearbeitet habe. Ich habe keine KI-Tools verwendet, deren Nutzung der Prüfer/ die Prüferin explizit schriftlich ausgeschlossen hat. Zusätzlich versichere ich, dass ich KI-gestützte Werkzeuge in der Rubrik „Übersicht verwendeter Hilfsmittel“ mit ihrem Produktnamen, meiner Bezugsquelle (z. B. URL) und Angaben zu genutzten Funktionen der Software sowie zum Nutzungsumfang vollständig aufgeführt habe. Weitere Vorgaben der Prüfer*innen zur Dokumentation habe ich entsprechend beachtet.

Hinweis: Sofern die zuständigen Prüfenden bis zum Zeitpunkt der Ausgabe der Aufgabenstellung konkrete KI-gestützte Schreibwerkzeuge ausdrücklich als nicht anzeige-/kennzeichnungspflichtig benannt haben, müssen diese nicht aufgeführt werden.

☐ **Option 3: Generativer KI nicht zugelassen und nicht verwendet**

Ich versichere, dass ich die vorliegende Arbeit vollständig eigenständig verfasst habe, also keine generativen KI-Tools oder Assistenzwerkzeuge verwendet habe.

Vincent Gentz
Dang
(Unterschriften)

Pforzheim, den 19.12.2025



S. Subari

Kurzfassung (Tom Liebegall)

Dieses Projekt beschäftigt sich damit, ein externes Parkassistenzsystem zu konzipieren und mit kostengünstigen IoT-Komponenten zu realisieren. Ziel ist eine flexible Alternative zu fest installierten Fahrzeugsensoren, die beim Parken durch visuelle Überwachung und Distanzmessung unterstützt. Das System nutzt eine Client-Server-Architektur. Zentrum der Erfassung ist ein ESP32-CAM-Modul, das Bilder über WLAN schickt. Die Auswertung erfolgt extern am Client mit dem Deep-Learning-Algorithmus YOLOv3, der Fahrzeuge zuverlässig erkennt und einordnet. Zusätzlich gibt es ein Ultraschall-Subsystem, das den Abstand zum Hindernis misst und dem Fahrer über eine LED-Leiste in drei Stufen klares, visuelles Feedback gibt. Der fertige Prototyp zeigt, dass so eine Lösung machbar ist und wie moderne KI-Verfahren gut mit Mikrocontrollern funktionieren.

Abstract (Daniel Egbe)

This project focuses on the design and implementation of an external parking assistance system using cost-effective IoT components. The aim is to provide a flexible alternative to permanently installed vehicle sensors, supporting the parking process through visual monitoring and distance measurement.

The system utilizes a client-server architecture. The core data acquisition unit is an ESP32-CAM module that transmits images via WiFi. Data processing is performed externally on the client side using the YOLOv3 deep learning algorithm, which reliably detects and classifies vehicles. Additionally, an ultrasonic subsystem measures the distance to obstacles and provides the driver with clear visual feedback in three stages via an LED strip. The final prototype demonstrates the feasibility of such a solution and illustrates the effective integration of modern AI methods with microcontrollers.

Inhalt

Abkürzungsverzeichnis	1
1 Einleitung (Tom Liebegall)	2
2 Problemstellung (Vincent Gentz)	3
2.1 Situation und Problemstellung	3
3 Ziele der Arbeit (Vincent Gentz)	4
3.1 Zielsetzung.....	4
3.2 Funktionale Anforderungen	4
3.3 Nicht-funktionale Anforderungen	4
3.4 Randbedingungen.....	4
4 Stand der Technik	6
4.1 Ultraschallsensor HC-SR04 (Daniel Egbe).....	6
4.1.1 Charakteristische Parameter.....	6
4.2 DS1307 (Serhat)	8
4.3 LCD1602 (Serhat).....	9
4.4 Identifikationstechnologien (Vincent Gentz)	10
4.4.1 HF-RFID und UHF-RFID.....	10
4.4.2 Bluetooth Low Energy (BLE).....	10
4.5 Datenhaltungskonzepte (Vincent Gentz)	11
4.5.1 Einfache Dateispeicherung (CSV/JSON).....	11
4.5.2 Eingebettete SQL-Datenbanken.....	11
4.5.3 Vergleiche	12
4.6 Polling- / Interrupt-Architektur (Vincent Gentz)	12
4.7 Sensorbasierte Verfahren zur Parkraumüberwachung (Tom Liebegall) ..	13
4.7.1 Optische Erfassung und maschinelles Lernen	13
4.7.2 Objekterkennung mittels YOLO-Algorithmus.....	13
4.7.3 Hardwareplattform ESP32-CAM mit OV2640	14
5 Konzeption	15
5.1 Grundkonzept (Vincent Gentz)	15
5.2 Konzepte der Abstandsmessung (Daniel Egbe).....	17
5.2.1 Abstandsmessung	17
5.2.2 Zeitliches Abtastkonzept.....	17
5.2.3 Logische Zonen-Definition	17
5.2.4 Warnlichter	18

5.3	Konzepte der Helligkeitssteuerung (Serhat Subasi).....	18
5.3.1	Zeitbasierte Steuerung.....	18
5.3.2	Stufenweise Dimmung	18
5.3.3	Helligkeitssensorbasierte Steuerung	18
5.3.4	Bewertung und Entscheidung	19
5.4	Möglichkeiten der Displayanzeige (Serhat Subasi).....	20
5.4.1	Ein- und Auscheckstatus	20
5.4.2	Anzeige der Parkdauer	20
5.4.3	Zeit- und Datumsanzeige.....	20
5.4.4	Permanente Timer- Anzeige	20
5.4.5	Informationsumschaltung.....	21
5.4.6	Auswahl.....	21
5.5	Digitale Dokumentation (Vincent Gentz)	21
5.6	Authentifizierung des Fahrers (Vincent Gentz)	21
5.7	Schnittstelle Datenbank/Arduino (Vincent Gentz).....	22
5.8	Systemarchitektur und Kommunikationsprotokoll (Tom Liebegall).....	22
5.8.1	Hardware-Selektion	23
5.8.2	Auswahl des Detektionsalgorithmus.....	24
6	Realisierung	25
6.1	Abstandsrealisierung (Daniel Egbe)	25
6.1.1	Elektrische Verschaltung und Pin-Belegung	25
6.1.2	Mechanischer Aufbau.....	25
6.1.3	Implementierung der Messung und Ansteuerlogik	26
6.1.4	Datentyp-Wahl.....	27
6.1.5	Realisierung der visuellen Feedback-Stufen	27
6.1.6	Hardware-Software-Kopplung der Anzeige	28
6.1.7	Verifikation durch Serielle Überwachung	29
6.2	Zeitbasierte Helligkeitssteuerung (Serhat Subasi)	29
6.3	Displayausgabe (Serhat Subasi).....	31
6.4	MFRC522 und Arduino-Software (Vincent Gentz)	35
6.5	Python Datenbank und GUI (Vincent Gentz).....	37
6.6	Python Schnittstelle (Vincent Gentz)	40
6.7	Implementierung der Firmware (ESP32) (Tom Liebegall).....	41
6.7.1	Bildverarbeitung und Objekterkennung (Python).....	42
6.7.2	Physischer Aufbau und Hardware-Optimierung.....	43
7	Zusammenfassung und Ausblick (Tom Liebegall)	45
7.1	Zusammenfassung	45

7.2	Ausblick	45
Literaturverzeichnis		47
Abbildungsverzeichnis		50
Tabellenverzeichnis		51
Listingverzeichnis		52
Anhang A Python-Code		53
A.1	Datei: database.py	53
A.2	Datei: gui.py	57
A.3	Datei: serial_listener.py	62
A.4	Datei: esp_IP-Kamera_Objekterkennung.py	64
Anhang B Arduino-Code		66
B.1	Main.ino	66
B.2	LCD.ino	70
B.3	Distance.ino	72
B.4	Leddim.ino	73
B.5	Updateled.ino	74
B.6	Datei: WIFICam.ino	75
Formelzeichenverzeichnis		77

Abkürzungsverzeichnis

API	Application Programming Interface (Schnittstelle zur Anwendungsprogrammierung)
CNN	Convolutional Neural Network (Faltendes neuronales Netzwerk)
DMA	Direct Memory Access (Direkter Speicherzugriff)
ESP	Espressif Systems (Hersteller des Mikrocontrollers)
FPS	Frames per Second (Bilder pro Sekunde)
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things (Internet der Dinge)
IoU	Intersection over Union (Schnittmenge über Vereinigungsmenge)
JPEG	Joint Photographic Experts Group (Bildformat)
NMS	Non-Maximum Suppression (Unterdrückung nicht-maximaler Werte)
WLAN	Wireless Local Area Network (Drahtloses lokales Netzwerk)
YOLO	You Only Look Once (Echtzeit-Objekterkennungsalgorithmus)
HF-RFID	High Frequency Radio-Frequency Identification
UHF-RFID	Ultra High Frequency Radio-Frequency Identification
BLE	Bluetooth Low Energy
SQL	Structured Query Language
GUI	Graphical User Interface

1 Einleitung (Tom Liebegall)

Die Anforderungen an die moderne Mobilität steigen ständig. Während Fahrzeuge durch fortschrittliche Technologien immer sicherer werden, bringt der Trend zu größeren Karosserien, besonders im SUV-Segment, neue Herausforderungen beim Parken mit sich. Die Übersichtlichkeit in den Fahrzeugen wird zwar durch vermehrte Sensoren unterstützt, doch die steigende Fahrzeugdichte in der Gesellschaft erschwert die Parkplatzsuche zusätzlich. Vor allem ältere Fahrzeuge ohne Assistenzsysteme sind hierbei benachteiligt.

Diese Arbeit beschäftigt sich daher mit der Konzeption und Entwicklung eines externen Parkassistentensystem, welches unabhängig von der Fahrzeugausstattung agiert und so die Sicherheit und Effizienz des Parkvorgangs für alle Verkehrsteilnehmer erhöht.

2 Problemstellung (Vincent Gentz)

2.1 Situation und Problemstellung

Im Zuge der Automobilentwicklung ist ein Trend zur Fertigung größerer und unübersichtlicher Fahrzeuge gewachsen [1]. Da gleichzeitig Parkassistenzsysteme, mit Kamera- und Ultraschall-Sensorik, zum Standard wurden, führt das bei Fahrzeugführern zu einer zunehmenden Abhängigkeit dieser verbauten Systeme. Ihre Funktionsfähigkeit ist nicht immer gegeben.

Erstens verfügen ältere Fahrzeugmodelle oder bestimmte Architekturen über keine Sensorik. Hinzu können bei Fahrzeugen, bei denen noch keine Modellpflege durchgeführt wurde, die Assistenzsysteme aufgrund von Softwarefehlern ausfallen. Eine Modellpflege dient zum Updaten und Korrigieren von Software und anderen Fehlern von einem Fahrzeug.

Zweitens kann die Hardware selbst eine Ausfallquelle darstellen. Die Hardware kann durch externe Einflüsse wie Wildunfälle oder aufgrund von Montagefehlern beschädigt oder fehlerhaft sein.

Drittens beeinträchtigen Umwelteinflüsse die Sensorik. Dazu gehören ungünstige Witterungsverhältnisse (Regen, Schnee, Nebel etc.), extreme Lichtbedingungen oder eine physische Bedeckung der Sensoren, beispielsweise durch Blätter [2].

In allen drei Szenarien ist das Parkassistenzsystem nicht in der Lage korrekte Messwerte zu liefern [2]. Folglich wird eine falsche Umgebung für den Fahrzeugführer dargestellt, sodass Fahrer aufgrund von Fahrzeugkonzept, Baujahr, technischen Defekten oder Umwelteinflüssen keinen sicheren Parkvorgang gewährleisten können.

3 Ziele der Arbeit (Vincent Gentz)

3.1 Zielsetzung

Das übergeordnete Ziel ist die Entwicklung eines intelligenten, externen Systems das den Parkvorgang umfassend digital unterstützt. Das System umfasst sowohl eine präzise und optische Signalisation des Abstands zwischen Fahrzeug und Parkplatzbegrenzung als auch ein Parkplatzmanagement. Letzteres erfasst Belegungsdaten in Echtzeit, Identifizierung der Nutzer und verwaltet Vorgänge über ein datenbasiertes Ein- und Auschecksystem, um unautorisierte Parkvorgänge zu lokalisieren.

3.2 Funktionale Anforderungen

Das System realisiert eine Echtzeit-Abstandsmessung zwischen Fahrzeug und Parkplatzbegrenzung mit optischer adaptiver Signalgebung. Die optische Signalisation passt sich automatisch an die jeweilige Tageszeit an, um die Wahrnehmung des Signals zu optimieren. Ein kontaktloses Authentifizierungs-System ermöglicht Nutzer-Check-in/out. Signalisiert wird ein Check-in/out durch eine Anzeige. Hinzu kommt eine Parkdaueranzeige beim Check-out. Ein unabhängiger Belegungssensor detektiert die Fahrzeugpräsenz. Bei fehlender Authentifizierung wird ein unautorisierter Parkvorgang klassifiziert. Sämtliche Daten werden für eine Analyse und Verwaltung gespeichert.

3.3 Nicht-funktionale Anforderungen

Für erfolgreichen Betrieb bildet ein zuverlässiges System, auf Software- / Hardwareebene die Grundlage. Darüber hinaus muss das gesamte System witterungsbeständig sein. Hinzu kommen eine intuitive Bedienoberfläche sowie eine modulare und skalierbare Architektur. Abschließend ist ein energieeffizienter Betrieb, für einen nachhaltigen Dauerbetrieb, notwendig.

3.4 Randbedingungen

Die systemtechnische Umsetzung basiert auf einem Arduino-Mikrokontroller und Sensoren-Integration. Die Mikrocontroller-Software wird in C++ implementiert, während für die strukturierte Nutzerdatenverwaltung auf eine relationsbasierte Datenspeicherung gesetzt wird. Zudem gewährleistet die Schnittstelle eine reibungslose Kommunikation zwischen Mikrokontroller und Nutzerdatenverwaltung.

Die Entwicklung selbst folgt einem modularen Arbeitsprinzip, bei dem jedes Teammitglied die Verantwortung für seinen klar definierten, eigenständigen Systemteilbereich trägt.

4 Stand der Technik

4.1 Ultraschallsensor HC-SR04 (Daniel Egbe)

Der HC-SR04 ist ein weit verbreitetes und kostengünstiges Modul zur Messung von Abständen im Nah- und Mittelbereich [3]. Er operiert nach dem Time-of-Flight-Prinzip, also der Messung der Zeit, die ein akustisches Signal für die Strecke der Sender zum Objekt und zurück benötigt [4].

Das verwendete Modul verfügt über einen Ultraschallsender (Transmitter) und einen Ultraschallempfänger (Receiver) [3]. Der Messvorgang läuft folgendermaßen ab. Über den Trigger-Pin wird ein Steuersignal mit einer Spannung von 5V und einer Mindestpulsdauer von 10µs angelegt, wodurch der Transmitter aktiviert wird. Das Modul sendet automatisch ein Signalkpaket mit acht Ultraschallimpulsen mit einer Frequenz von 40kHz aus. Die ausgesendeten Ultraschallwellen breiten sich mit der Schallgeschwindigkeit aus. Treffen sie auf ein Hindernis, werden sie reflektiert und vom Empfänger detektiert. Mit Beginn des Sendevorgangs wechselt der Echo-Pin in den High-Zustand bis zum Empfang der reflektierten Welle und wechselt dann in den Low-Zustand. Die Zeit t dieser High-Phase entspricht der Laufzeit des Signals [3].

Das Berechnen der Distanz d erfolgt mithilfe der bekannten Schallgeschwindigkeit c_{Schall} (ca. 343 m/s). Da die Zeit t die Laufzeit für den Hin- und Rückweg einschließt, muss sie halbiert werden:

$$d = \frac{t \cdot c_{Schall}}{2} \quad (1)$$

4.1.1 Charakteristische Parameter

Messbereich: Der typische Messbereich des HC-SR04 liegt zwischen 2 cm und 400 cm. Diese Distanz erfüllt die Anforderungen des Parkassistenzsystems vollständig, insbesondere im Nah- und mittleren Distanzbereich [3]. Die Messgenauigkeit des Sensors wird vom Hersteller mit ± 3 mm angegeben. Diese Genauigkeit ist für die vorgesehene Anwendung ausreichend. Die Messgenauigkeit des Sensors ist maßgeblich von den Umgebungsbedingungen abhängig [5]. Insbesondere Temperatur und Luftfeuchtigkeit beeinflussen die Schallgeschwindigkeit und wirken sich somit direkt auf die Distanzberechnung aus. Darüber hinaus kann es bei schallabsorbierenden Materialien, wie beispielsweise Textilien, zu einer

Abschwächung des reflektierten Signals kommen. Ebenso können stark geneigte Oberflächen zu einer Totalreflexion der Ultraschallwellen führen, wodurch fehlerhafte oder ausbleibende Messwerte entstehen können.

Tabelle 1: Vergleich Sensoren

Sensorik	Prinzip	Messbereich	Kosten/Komplexität	Störanfälligkeit
Ultraschall (HC-SR04)	Laufzeitmessung (akustisch)	Kurz bis Mittel (bis 4 Metern)	Niedrig/Gering	Temperatur, Weiche Materialien
Infrarot-Sensorik (IR)	Triangulation (optisch)	Sehr kurz bis Mittel (bis 1,5 Metern)	Niedrig/Gering	Umgebungslicht, Oberflächenfarbe
LiDAR	Laufzeitmessung (optisch/Laser)	Mittel bis Weit (bis über 100 m)	Hoch/Hoch	Rauch, Nebel, Regen

In Tabelle 1 wird der Ultraschallsensor mit anderen Sensoren verglichen [5]. Die Entscheidung für den HC-SR04 Ultraschallsensor basiert einerseits auf den Projektanforderungen, andererseits spielen die verfügbaren Ressourcen eine große Rolle. Der HC-SR04 stellt im Gegensatz zu den konkurrierenden Sensoren (Tabelle 1) den optimalen Kompromiss aus Funktionalität, einfacher Bedienung und niedrigen Kosten für die Realisierung des Projekts dar.

4.2 DS1307 (Serhat)

Bei vielen zeitabhängigen Mikrocontroller-Projekten kommen Echtzeituhren zum Einsatz. Sie vereinfachen die Integration von Uhrzeiten und Kalendertagen. Die Echtzeituhr (RTC) DS1307 ist eine der bekanntesten. Sie stellt Informationen über Sekunden, Minuten, Stunden, Wochentage, das Datum, den Monat und das Jahr bereit. Die Anzahl der Tage eines Monats sowie Schaltjahre werden automatisch angepasst [6]. Der DS1307 kommuniziert über den I²C-Bus. Dadurch kann sie praktisch mit jedem modernen Mikrocontroller verbunden werden. Außerdem ist die RTC batteriegepuffert, sodass nach einmaligem Einstellen auch nach Unterbrechung der Stromversorgung die Uhrzeit mit derselben Genauigkeit zur Verfügung gestellt wird [7]. Die RTC unterstützt das 24-Stunden- und das 12-Stunden-Format (AM/PM) [6]. Sie arbeitet mit binär codierten Dezimalzahlen: Dabei wird jede einzelne Ziffer einer Dezimalzahl durch vier Bits dargestellt. Die Dezimalzahlen 0 bis 9 entsprechen binär codiert den Zahlen 0000 bis 1001, was sich gut eignet, um beispielsweise Werte von Minuten zwischen 00 und 59 oder von Monaten zwischen 1 und 12 zu speichern [6] [8]. Somit eignet sich der DS1307 gut, um die Helligkeit der Anzeige zeitabhängig zu steuern. Eine Alternative wäre der DS3231, der genauer, aber auch teurer ist [9]. Für die funktionalen Anforderungen, die Helligkeit der Anzeige zu steuern und die Parkzeit zu ermitteln, reicht jedoch die Genauigkeit des DS1307 aus.

4.3 LCD1602 (Serhat)

Textbasierte Displayanzeigen stellen ein geeignetes Mittel zur Bereitstellung von Rückmeldungen an Nutzer dar. Weit verbreitet ist dabei das LCD1602-Modul (Liquid Crystal Display), ein alphanumerisches Punktmatrix-Flüssigkristalldisplay mit zwei Zeilen zu jeweils 16 Zeichen [10] [11]. Die Grundlage bildet der HD44780-LCD-Controller, der die komplette Zeichendarstellung und Ansteuerung der LCD-Matrix übernimmt. Jedes Zeichen wird in einer 5 x 8-Punktmatrix dargestellt. Der HD44780 verfügt über ein 80-Byte-DDRAM sowie ein CGROM und ein CGRAM, die für Standard- und benutzerdefinierte Zeichen zur Verfügung stehen [10]. Das LCD-Modul hat jedoch keine I²C-Schnittstelle, weshalb es über einen I²C-Adapter mit dem Mikrokontroller verbunden wird. Dieser sorgt dafür, dass statt 16 Leitungen nur vier benötigt werden. Der I²C-Adapter verfügt über integrierte Einstellmöglichkeiten für den Kontrast und die Hintergrundbeleuchtung [12]. Diese Eigenschaften und Funktionen ermöglichen die Ausgabe einer Rückmeldung bei erfolgreichem Einchecken und der Parkdauer auf einem Display. Alternativen wie OLED- oder TFT-Displays haben eine höhere Auflösung und ermöglichen eine grafische Darstellung. Sie sind jedoch auch kostenintensiver und bieten keinen größeren Nutzen.

4.4 Identifikationstechnologien (Vincent Gentz)

4.4.1 HF-RFID und UHF-RFID

HF-RFID-Systeme (High Frequency) nutzen die induktive Kopplung zur Energie- und Datenübertragung. Diese Systeme operieren im Frequenzbereich von 3 bis 30 MHz. Das im Lesegerät erzeugte magnetische Wechselfeld induziert eine Spannung im RFID-Transponder, welche den Mikrochip versorgt und die Datenübertragung mittels Lastenmodulation ermöglicht. Die typische Lesereichweite dieser Technologie liegt bei weniger als 1 Meter [13].

UHF-RFID-Systeme (Ultra High Frequency) basieren auf der elektromagnetischen Kopplung zur Energie- und Datenübertragung und operieren im Frequenzbereich von 200 MHz bis 2GHz. Passive UHF-Transponder beziehen ihre Energie über die vom Lesegerät ausgesendeten elektromagnetischen Wellen und übertragen die Daten durch Reflexion (Backscatter). Die Lesereichweiten innerhalb der EU liegen bei 3 bis 5 Meter, abhängig von der Sendeleistung, in den USA beträgt diese 5 bis 7 Meter unter idealen Bedingungen. Semi-aktive Systeme erreichen Reichweiten von maximal 15 Meter und aktive Systeme bis zu 100 Meter. Allerdings sind UHF-RFID fehleranfälliger gegenüber Umwelteinflüssen im Vergleich zu HF-RFID [13].

4.4.2 Bluetooth Low Energy (BLE)

Bluetooth Low Energy (BLE) ist eine drahtlose, energieeffiziente Technologie für die Kurzstreckenkommunikation. Sie operiert im 2,4GHz ISM-Band eine Datenrate von bis zu 1 Mbps. Die Datenübertragung erfolgt über Advertising- und Data-Channels. Dabei wird Frequency Hopping eingesetzt, bei dem das Signal innerhalb jeder Übertragung auf unterschiedliche Frequenzkanäle springt, um Interferenzen zu vermeiden. Die hohe Energieeffizienz wird durch das Senden der Daten innerhalb kurzer aktiver Sendeintervalle gewährleistet, außerhalb dieses Intervalls begibt sich der Sender in den energiearmen Schlafmodus. Die typische Reichweite beträgt mehrere zehn Meter, abhängig von der Sendeleistung und Umwelteinflüssen [14].

4.5 Datenhaltungskonzepte (Vincent Gentz)

4.5.1 Einfache Dateispeicherung (CSV/JSON)

CSV-Dateien sind textbasierte Dokumente zur Speicherung tabellarisch strukturierter Daten. Die Strukturierung erfolgt gemäß RFC 4180 [15] durch die Separierung der einzelnen Werte mittels festgelegter Trennzeichen. Die Abgrenzung der Datensätze erfolgt mit CRLF als Zeilenumbruch. Optional kann die Kopfzeile zur Bezeichnung der Datenfelder in die erste Zeile eingefügt werden.

JSON-Dateien hingegen repräsentieren **digitale** Strukturen, wie in RFC 8259 definiert [16]. Das Format basiert auf einer Kombination aus vier primitiven Typen (Strings, Numbers, Booleans und null) sowie zwei Strukturen (Objekte und Arrays). Die Manipulation dieser Dokumente wird durch den Einsatz von Parser und Generatoren ermöglicht [16].

Beide Dateiformate ermöglichen die Darstellung einfacher Datenstrukturen und die Bearbeitung mit Hilfe von Programmiersprachen. Die Suchkomplexität ist abhängig von der konkreten Implementierung und dem Suchalgorithmus. Standardmäßig wird eine sequenzielle Verarbeitung ohne spezielle Indizierung verwendet. Implizit folgt daraus eine lineare Suche mit Worst-Case-Zeitkomplexität $O(n)$, mit n als Anzahl der Datensätze [17].

4.5.2 Eingebettete SQL-Datenbanken

Eine Datenbank, deren Struktur dem relationalen Modell folgt, wird über die Sprache SQL (Structured Query Language) verwaltet. Die Struktur basiert auf Relationen, wobei die Datenhaltung in Tabellen erfolgt, welche aus Attributen (Spalten) und Tupeln (Zeilen) besteht. Attribute haben eindeutige Namen und festgelegte Datentypen. Jedes Tupel wird über einen Primärschlüssel (PRIMARY KEY) identifizierbar. Tabellen können mittels der vier grundlegenden CRUD-Operationen (create, read, update, delete) implementiert, manipuliert und abgefragt werden. Die zugehörigen SQL-Befehle sind INSERT, SELECT, UPDATE und DELETE. Durch Primärschlüssel erfolgt eine automatische Indizierung, die auf einem B+-Baum basiert. Folglich weisen Suchoperationen in relationalen Datenbanken eine logarithmische Zeitkomplexität $O(\log(n))$ auf [18] [19].

4.5.3 Vergleiche

Die Datenspeicherung kann lokal (On-Premises) oder in einer Cloud-Umgebung erfolgen. Die lokale Implementierung impliziert eine vollständige Eigenverwaltung des eigenen Rechenzentrums, einschließlich Wartung, Backup und Recovery [20].

Dies führt zu signifikant höheren Betriebskosten und Investitionskosten implizit folgt daraus eine Reduzierung der Flexibilität und Skalierbarkeit [20]. Im Cloud-Modell hingegen wird die Verwaltung vom Dienstleister übernommen, wodurch die Kosten primär als laufende Betriebskosten wahrgenommen werden und die Datenhaltung dynamisch skaliert werden kann [20]. Während das Sicherheitskonzept bei On-Premises eigenständig implementiert werden muss, erfordert das Cloud-Modell Vertrauen beim Dienstleister [20].

4.6 Polling- / Interrupt-Architektur (Vincent Gentz)

Eine Polling-Architektur, synchrone I/O, stellt einen Prozessablauf dar, bei dem die Zentraleinheit den Status eines technischen Objektes oder dessen Datenpuffer aktiv und periodisch abfragt.

Im Gegensatz sendet die Interrupt-Architektur, asynchrone I/O, selbständig ein Interrupt-Signal an die Zentraleinheit, sobald ein Ereignis eingetreten ist. Eine permanente Statusüberprüfung fällt weg [21].

4.7 Sensorbasierte Verfahren zur Parkraumüberwachung (Tom Liebegall)

Zur Detektion von Fahrzeugen auf Stellflächen kommen in der Praxis häufig induktive, optische oder akustische Sensoren zum Einsatz. Weit verbreitet sind Induktionsschleifen oder Hall-Sensoren, die Änderungen im Magnetfeld erfassen, sobald sich eine metallische Karosserie darüber befindet [22]. Diese Systeme gelten als robust gegenüber Witterungseinflüssen, erfordern jedoch häufig Eingriffe in die Bodeninfrastruktur, was die Installation aufwendig und kostenintensiv gestaltet.

Alternativ werden Infrarot- oder Ultraschallsensoren eingesetzt. Diese messen Distanzen oder Signalunterbrechungen, um die Belegung eines Parkplatzes zu prüfen. Problematisch scheint hier vor allem die Störanfälligkeit: Infrarotsensoren können durch starke Sonneneinstrahlung beeinträchtigt werden, während Ultraschallsensoren bei ungünstigen Reflexionswinkeln ungenaue Messwerte liefern [23]. Ein wesentlicher Nachteil dieser konventionellen Methoden ist zudem die fehlende Klassifikationsfähigkeit. Ein Sensor registriert lediglich die Anwesenheit eines Objekts, kann jedoch nicht differenzieren, ob es sich tatsächlich um ein Kraftfahrzeug oder ein anderes Hindernis handelt [22].

4.7.1 Optische Erfassung und maschinelles Lernen

Im Gegensatz zu einfachen Näherungssensoren bieten kamerabasierte Systeme eine höhere Informationsdichte. Durch den Einsatz von Computer Vision (maschinelles Sehen) können erfasste Bilddaten nicht nur auf Bewegung, sondern auch semantisch analysiert werden [24]. Dies ermöglicht eine zuverlässige Unterscheidung zwischen verschiedenen Objektklassen.

Während klassische Bildverarbeitungsmethoden oft auf statischen Hintergrundmodellen basieren, haben sich in jüngster Zeit Verfahren des Deep Learning durchgesetzt. Hierbei werden künstliche neuronale Netze (CNNs) trainiert, um spezifische Merkmale von Fahrzeugen in unterschiedlichen Umgebungen und Perspektiven robust zu erkennen [24].

4.7.2 Objekterkennung mittels YOLO-Algorithmus

Für Echtzeit-Anwendungen in der Parkraumüberwachung ist die Verarbeitungsgeschwindigkeit von entscheidender Bedeutung. Hier hat sich der YOLO-Algorithmus („You Only Look Once“) als Standard etabliert. Im Gegensatz zu älteren Verfahren, die ein Bild in mehreren Schritten analysieren, betrachtet YOLO

das Bild in einem einzigen Durchlauf eines neuronalen Netzes [25]. Das Bild wird in ein Raster unterteilt, und für jede Rasterzelle werden gleichzeitig Begrenzungsrahmen (Bounding Boxes) und Wahrscheinlichkeiten für verschiedene Klassen (z. B. „Auto“, „Person“) berechnet.

Diese Architektur ermöglicht eine sehr schnelle Inferenz, was YOLO besonders geeignet für Anwendungen macht, bei denen eine sofortige Rückmeldung erforderlich ist [25]. Da die Ausführung komplexer neuronaler Netze hohe Rechenleistung erfordert, wird in vielen IoT-Szenarien ein Edge-Computing-Ansatz verfolgt: Ein Mikrocontroller erfasst das Bild, während die rechenintensive Auswertung auf eine leistungstärkere Einheit, wie einen Laptop oder Server, ausgelagert wird [26].

4.7.3 Hardwareplattform ESP32-CAM mit OV2640

Als Schnittstelle zwischen der physischen Parkumgebung und der digitalen Bildverarbeitung dient in diesem Projekt das Entwicklungsboard ESP32-CAM. Hierbei handelt es sich um eine kosteneffiziente IoT-Plattform, die auf dem ESP32-S-Chip basiert und standardmäßig mit einem OV2640 Kameramodul ausgestattet ist [27].

Das verwendete Modul verfügt über eine integrierte 2,4 GHz Wi-Fi- und Bluetooth-Schnittstelle, was eine kabellose Übertragung der Bilddaten ermöglicht. Der OV2640-Sensor bietet eine Auflösung von bis zu 2 Megapixeln (UXGA 1622×1200), was für Objekterkennungsaufgaben in Nahbereichsanwendungen ausreichend ist, ohne die Bandbreite bei der Übertragung unnötig zu belasten. Ein weiterer Vorteil dieser spezifischen Hardwarekonfiguration ist die Möglichkeit, eine externe 2,4 G Antenne anzuschließen, um die Signalstabilität auch bei schwierigen Abschirmungsbedingungen (z. B. in Garagen) zu gewährleisten [27]. Da das Board mit 5 V betrieben werden kann, lässt es sich unkompliziert in bestehende Mikrocontroller-Setups integrieren.

5 Konzeption

5.1 Grundkonzept (Vincent Gentz)

Das primäre Ziel des Konzepts ist die Entwicklung eines externen Parkassistenzsystem, das Fahrer eines Kraftfahrzeugs unter suboptimalen Bedingungen assistiert. Die Basis bildet die Distanzmessung, welche den Abstand zwischen dem Kraftfahrzeug und der Parkplatzbegrenzung in Echtzeit erfasst. Die erfasste Distanz wird ergänzend über eine Distanzanzeige visualisiert. Diese ist so positioniert, dass sie beim Rückwärtseinparken über die Seitenspiegel und beim Vorwärtsparken über die Frontscheibe einsehbar ist. Die Farbcodierung basiert auf definierten Schwellenwerten, die dem Fahrer den Abstand in drei Stufen signalisiert. Hierbei wird die Farbe Orange als optimales Abstandssignal verwendet, dass das Zielintervall kennzeichnet. Die Farbe Grün signalisiert, dass der Abstand größer als 1 Meter beträgt, orange bei einem Abstand zwischen 1 Meter und 0,5 Meter und rot, wenn der Abstand weniger als 0,5 Meter beträgt, wie in Tabelle 2 beschrieben.

Tabelle 2: Abstand-Farbcodierung

ABSTAND (IN METER)	FARBCODIERUNG
> 1	Grün
< 1 UND > 0,5	Orange
< 0,5	Rot

Die Distanzanzeige wird durch eine Parkplatzbelegungserkennung ergänzt. Sie erfasst das physische Kraftfahrzeug, falls dieses auf dem Parkplatz steht.

Hinzu erfolgte eine Erweiterung der Infrastruktur für einen Privatparkplatz durch ein Ein-/Auscheck-System. Dieses System bietet Bewohner oder Anliegern die Möglichkeit, sich nach dem erfolgreichen Parkvorgang bzw. vor dem Ausparken zu authentifizieren. Das Authentifizierungsmodul verfügt über eine Anzeige, die zur Darstellung relevanter Systeminformationen wie Parkdauer sowie erfolgreiches Authentifizieren für den Endnutzer dient.

Die Kombination aus der Parkplatzbelegungserkennung und dem Authentifizierungssystem dient zur Identifizierung unautorisierter Parkvorgänge. Zur Gewährleistung der Auditierbarkeit aller Vorgänge, wird eine digitale und persistente

Protokollierung implementiert. Die Administratoren verfügen über eine grafische User Interface (GUI) der aktuell protokollierten Vorgänge. Die technische Umsetzung des Konzepts geht von einem Privatparkplatz aus.

5.2 Konzepte der Abstandsmessung (Daniel Egbe)

5.2.1 Abstandsmessung

Zur Abstandsmessung wurde der bereits erwähnte Ultraschallsensor HC-SR04 eingesetzt. Der Sensor verfügt über einen Messbereich von 2 cm bis 400 cm, welcher für die Anforderungen dieses Projekts geeignet ist. Die Distanzmessung erfolgt durch das periodische Aussenden von Ultraschallimpulsen und die anschließende Erfassung der Laufzeit des reflektierten Signals. Unter Berücksichtigung der Schallgeschwindigkeit sowie der vom Sensor gemessenen Signallaufzeit kann der Abstand zwischen dem Kraftfahrzeug und der Parkbegrenzung berechnet werden.

5.2.2 Zeitliches Abtastkonzept

Bei der Konzeption des Parkassistenzsystems ist eine Reaktionszeit des Fahrzeugführers zu berücksichtigen. Ein großes Zeitintervall zwischen aufeinanderfolgenden Messungen kann zu verzögerten Warnsignalen während des Einparkvorgangs führen. Aus dem Grund wurde ein Messintervall von 14 μ s festgelegt. Dieses Intervall stellt sicher, dass selbst bei niedriger Fahrgeschwindigkeit im Bereich der Schrittgeschwindigkeit zwischen zwei Messungen lediglich eine vernachlässigbare Wegstrecke zurückgelegt wird. Dadurch ist eine präzise Abstandserfassung im Nahbereich gewährleistet.

5.2.3 Logische Zonen-Definition

Das zentrale Konzept besteht darin, kontinuierlich erfasste Distanzwerte in diskreten Warnstufen darzustellen. Zu diesem Zweck wird ein dreistufiges Zonenmodell definiert, das als Grundlage für die Ansteuerung der LED-Warnanzeige dient.

Zone 1 umfasst Entfernungen von 100 cm und größer. In diesem Bereich ist keine Warnung erforderlich, da sich das Kraftfahrzeug in einem ausreichenden Abstand zur Parkbegrenzung befindet und keine Kollisionsgefahr besteht. Die LED-Anzeige leuchtet in diesem Fall grün.

Zone 2 wird über einen Distanzbereich von 50 cm bis 100 cm definiert. In dieser Zone wird eine Annäherung an die Parkbegrenzung detektiert. Entsprechend nimmt die Warnintensität mit abnehmender Distanz zu, wobei die LED-Farbe von grün auf Gelb wechselt.

Zone 3 umfasst 50 cm und darunter. In diesem Bereich liegt kritischer Abstand vor. Zur Vermeidung einer Kollision wird eine deutliche Warnung ausgegeben, wobei die LED-Anzeige auf Rot umschaltet.

5.2.4 Warnlichter

Als optisches Warnsignal zur Kollisionsvermeidung wurde die direkt auf dem Arduino integrierte LED (NeoPixel LED-Leiste) verwendet, da im Vergleich zu externen LEDs, wie beispielsweise RGB-Leuchten, keine zusätzlichen Widerstände oder externe Anschlüsse erforderlich sind, wodurch der Aufwand sowie der Stromverbrauch reduziert werden [28]. Zudem weist die integrierte LED eine hohe Leuchtintensität auf, wodurch eine klare und gut wahrnehmbare visuelle Warnung gewährleistet ist [28].

5.3 Konzepte der Helligkeitssteuerung (Serhat Subasi)

Für die Helligkeitssteuerung der optischen Rückmeldung (LED) wurden mehrere umsetzbare Konzepte untersucht. Diese basieren entweder auf einer zeitabhängigen Steuerung über die Echtzeituhr (DS1307) oder auf sensorgestützten Ansätzen.

5.3.1 Zeitbasierte Steuerung

Mithilfe der DS1307 kann die Helligkeit zu definierten Uhrzeiten automatisch angepasst werden, beispielsweise durch eine Reduzierung ab 20:00 Uhr und eine Erhöhung ab 08:00 Uhr. Ebenso lassen sich tagesabhängige Regeln umsetzen, etwa das vollständige Abschalten der LED an Sonn- und Feiertagen. In solchen Fällen erscheint auf dem Display eine entsprechende Meldung, die darüber informiert, dass der Parkvorgang nicht möglich ist. Dieses Konzept zeichnet sich durch hohe Zuverlässigkeit und geringe Störanfälligkeit aus, da es unabhängig von den äußeren Lichtverhältnissen funktioniert.

5.3.2 Stufenweise Dimmung

Eine weitere Möglichkeit ist eine schrittweise Helligkeitsänderung über einen definierten Zeitraum. Dadurch entstehen weiche Übergänge, was den visuellen Komfort verbessert. Allerdings ist dieses Konzept praktisch nur sinnvoll, wenn die Dimm-Phasen mit der natürlichen Dämmerung synchronisiert werden. Aufgrund der stark variierenden Dämmerungszeiten je nach Saison ist eine zuverlässige Umsetzung ohne zusätzliche Sensorik schwierig.

5.3.3 Helligkeitssensorbasierte Steuerung

Ein Helligkeitssensor ermöglicht eine dynamische Anpassung der LED an wechselnde Umgebungslichtbedingungen. Allerdings besteht das Risiko von Fehlsteuerungen, beispielsweise durch Verschmutzung, Abschattung oder ungünstige Reflexionen. Daher bietet dieses Konzept im Vergleich zur zeitbasierten Steuerung eine geringere Betriebszuverlässigkeit.

5.3.4 Bewertung und Entscheidung

Der Helligkeitssensor bietet zwar eine hohe Adaptivität, ist jedoch störanfällig. Auch die stufenweise Dimmung bei einer natürlichen Dämmerung ist schwer realisierbar. Die zeitbasierte Steuerung über die Echtzeituhr hingegen ist technisch einfach umzusetzen, robust gegenüber Umweltfaktoren und vollkommen ausreichend zur Erfüllung der Anforderung. Zusätzlich übernimmt die Echtzeituhr bereits eine weitere Aufgabe im System etwa die Berechnung der Parkdauer, wodurch kein zusätzliches Bauteil benötigt wird.

Aus diesen Gründen wurde die zeitbasierte Helligkeitssteuerung über die Echtzeituhr zu festgelegten Uhrzeiten ausgewählt. Sie bietet ein optimales Verhältnis aus Funktionalität, Zuverlässigkeit, Kosten und Wartungsaufwand.

5.4 Möglichkeiten der Displayanzeige (Serhat Subasi)

Für die visuelle Rückmeldung des Systems wurden verschiedene Lösungskonzepte untersucht. Ziel war es, eine Anzeigeform zu definieren, die dem Nutzer klare und relevante Informationen liefert, ohne das Display zu überladen oder die Bedienung unnötig zu komplizieren. Dabei standen die Nutzerfreundlichkeit und die funktionale Relevanz im Fokus

5.4.1 Ein- und Auscheckstatus

Die Anzeige des Ein- und Auscheckstatus bietet dem Nutzer eine unmittelbare Bestätigung darüber, dass sein RFID-Tag korrekt erkannt wurde und der Parkvorgang entsprechend gestartet oder beendet ist. Dadurch wird transparent signalisiert, dass der Eintrag in der Datenbank erfolgt ist und die Erfassung der Parkzeit korrekt begonnen bzw. abgeschlossen wurde. Diese Form der Rückmeldung ist für die Nutzerfreundlichkeit essenziell und stellt den Kernprozess des Systems klar dar.

5.4.2 Anzeige der Parkdauer

Nach Beendigung des Parkvorgangs wird die berechnete Parkdauer für wenige Sekunden eingeblendet. Diese Information ist für den Nutzer besonders relevant, da sie eine unmittelbare Rückmeldung über die tatsächliche Dauer des Parkplatzaufenthalts liefert. Die Anzeige erfolgt zeitlich begrenzt, damit das Display anschließend wieder in den Grundzustand wechseln kann. Die Umsetzung ist technisch unkompliziert und nutzt praktisch die bereits im System vorhandene Echtzeituhr, wodurch zusätzliche Hardwarekomponenten entfallen.

5.4.3 Zeit- und Datumsanzeige

Die permanente Anzeige von Uhrzeit und Datum wurde in Betracht gezogen. Da dies jedoch keinen direkten funktionalen Bezug zum Parkvorgang aufweist, wurde diese Idee verworfen.

5.4.4 Permanente Timer- Anzeige

Die fortlaufende Anzeige der aktuellen Parkdauer während des Parkvorgangs wurde ebenfalls in Erwägung gezogen. Wurde jedoch nicht gewählt da die kontinuierliche Anzeige der Parkdauer nach dem Einparken keinen Mehrwert bietet.

5.4.5 Informationsumschaltung

Die zyklische Darstellung unterschiedlicher Inhalte wie Statusinformation, Uhrzeit, Parkdauer oder Systemmeldungen wurde analysiert. Der Ansatz bietet den Vorteil, eine Vielzahl von Informationen auf begrenztem Anzeigeraum darstellen zu können. Gleichzeitig besteht jedoch die Gefahr einer verminderten Übersichtlichkeit. Darüber hinaus kann es erforderlich sein, dass Nutzer auf das erneute Erscheinen der jeweils relevanten Information warten müssen, was sich negativ auf die Benutzerfreundlichkeit auswirkt.

5.4.6 Auswahl

Schlussendlich wurde die Kombination aus der Anzeige für das ein- und Auschecken mit dem der Anzeige der Parkdauer gewählt. Die Parkdauer wird nach der Anzeige der Bestätigung für das Auschecken für 5 Sekunden angezeigt. Dieses Konzept vermittelt die wichtigsten Informationen klar und direkt und überlastet den Nutzer nicht mit unnötigen Daten. Zudem ist das auf dem kleinen Display gut darstellbar. Damit erfüllt die gewählte Anzeigeform die funktionale Anforderung und hält das System gleichzeitig übersichtlich und robust.

5.5 Digitale Dokumentation (Vincent Gentz)

Für die Speicherung der Daten des Parkassistenzsystems wurde eine lokale SQL-Datenbank gewählt. Dies bietet im Vergleich zu einfachen Dateiformaten wie CSV oder JSON eine höhere Abfrageeffizienz. Durch Verwendung von Primärschlüsseln wird eine Indizierung eingeleitet, welche die Suchkomplexität auf $O(\log n)$ reduziert, während die sequenzielle Suche bei Dateiformaten $O(n)$ aufweist. Hierbei ist die sequenzielle Suche ein standardisiertes Suchverfahren.

Zudem erlaubt eine relationale Datenbank eine klare Verknüpfung zwischen Chip/Karte, Parkberechtigter und Check-in-/Check-out-Daten, was die Verifizierung vereinfacht. Der lokale Aspekt der Datenbank gewährleistet, dass sämtliche Nutzerdaten unter eigener Kontrolle bleiben.

5.6 Authentifizierung des Fahrers (Vincent Gentz)

Für das Einchecken am Parkplatz wurde die HF-RFID Technologie gewählt. Die Begründung dafür liegt in der sehr kurzen Reichweite dieser Technologie. Sie gewährleistet, dass beim Ein- oder Auschecken eine aktive Nähe des Fahrers vorausgesetzt wird, um das Lesegerät auszulösen. Alternativen wie UHF-RFID oder

Bluetooth Low Energy (BLE) sind aufgrund der größeren Reichweiten ungeeignet für das Projekt, da durch Vorbeifahren unbeabsichtigt ein- oder ausgecheckt werden kann. Die HF-RFID Technologie ist für das Parkassistenzsystem die ideale Lösung.

5.7 Schnittstelle Datenbank/Arduino (Vincent Gentz)

Für die Implementierung der Schnittstelle zur Datenübertragung vom Arduino-System zur Datenbank wurde die Polling-Architektur gewählt. Die Polling-Architektur garantiert eine feste Reihenfolge von Ereignissen und ermöglicht dadurch eine sequenzielle Abarbeitung. Dieser deterministische Ablauf ist für die Entwicklung robuster Echtzeit-Listener essenziell, da unvorhersehbare Kontextwechsel vermieden werden. Polling ist ideal für die Verarbeitung kleiner, häufiger Datenpakete mit geringer Wartezeit und führt zu einer niedrigeren Implementierungskomplexität der Schnittstellenlogik, da der zusätzliche Overhead entfällt. Trotz eines erhöhten Energieverbrauchs bei der Polling-Architektur, fällt dieser Nachteil in der Implementierung weg, da das Parkassistenzsystem auf einem eigenen, dedizierten Hardware-System ohne konkurrierende Prozesse läuft.

Die Kombination aus Polling in einer Python-Umgebung resultiert in einer leicht wartbaren Schnittstellenlösung (Listener), welche die Daten des Systems aktiv ausliest und in die Datenbank schreibt.

5.8 Systemarchitektur und Kommunikationsprotokoll (Tom Liebegall)

Aufgrund der physikalischen Gegebenheiten in Parkhäusern (Abschirmung, Distanz) wurde eine verteilte Systemarchitektur gewählt. Das System unterteilt sich in die *Datenerfassungsebene* (Sensorik) und die *Verarbeitungsebene* (Logik).

Für den Datentransfer wurde bewusst auf komplexe Streaming-Protokolle wie RTSP (Real-Time Streaming Protocol) verzichtet. Stattdessen kommt ein Polling-Verfahren über HTTP (Hypertext Transfer Protocol) zum Einsatz.

- **Begründung:** Ein Parkvorgang ist ein kinematischer Prozess mit geringer Dynamik. Eine Bildwiederholrate von 1–2 FPS ist für die Statuserkennung ("Belegt" vs. "Frei") ausreichend.
- **Vorteil:** HTTP ist zustandslos (stateless). Sollte die WLAN-Verbindung kurzzeitig abbrechen, führt der nächste Request des Clients automatisch zur

Wiederaufnahme der Übertragung, ohne dass ein Videostream neu ausgehandelt werden muss. Dies erhöht die Ausfallsicherheit signifikant.

5.8.1 Hardware-Selektion

Für die Erfassungseinheit ("Edge Device") wurden verschiedene Mikrocontroller und Einplatinencomputer verglichen. Entscheidende Kriterien waren die Startzeit (Boot-Time), der Energiebedarf und die Integrationsfähigkeit einer Kamera. **Tabelle 3** fasst die Entscheidungsmatrix zusammen.

Tabelle 3: Technischer Vergleich der Hardware-Optionen

Kriterium	ESP32-CAM	Raspberry Pi Zero W	Bewertung
Kosten	< 10 € (inkl. Kamera)	ca. 25 € (+ Kamera-Modul)	ESP32 ermöglicht kosteneffiziente Skalierung
Boot-Zeit	< 2 Sekunden (RTOS)	> 30 Sekunden (Linux OS)	ESP32 ist sofort einsatzbereit (Ad-hoc)
Energiebedarf	ca. 180 mA (WLAN aktiv)	ca. 400 mA (Idle)	ESP32 ist energieeffizienter
Komplexität	Gering (Firmware)	Hoch (Betriebssystem)	ESP32 minimiert Wartungsaufwand (kein OS-Update nötig)

Ein kritisches Kriterium in der Konzeptionsphase war die Signalintegrität. Da das ESP32-Modul oft in Gehäusen oder hinter Betonwänden verbaut wird, reicht die integrierte PCB-Antenne (Gain < 2 dBi) oft nicht aus. Das Konzept schreibt daher zwingend die Verwendung eines Moduls mit IPEX-Anschluss für eine externe 2,4 GHz Stabantenne vor, um auch bei niedrigem RSSI (Received Signal Strength Indicator) eine stabile Bildübertragung zu gewährleisten.

5.8.2 Auswahl des Detektionsalgorithmus

Die Kernkomponente der Verarbeitungsebene ist das Convolutional Neural Network (CNN). Zur Auswahl standen die Architekturen *YOLOv3* und *YOLOv3-Tiny*. Die Entscheidung fiel zugunsten des Standard-*YOLOv3*-Modells, obwohl dieses rechenintensiver ist (Tabelle 4). Der technische Hintergrund liegt in der Feature-Extraction-Tiefe:

- **YOLOv3-Tiny** nutzt eine reduzierte Architektur mit nur 33 Layern. Dies führt dazu, dass kleine Objekte (z. B. Autos im Hintergrund des Parkplatzes) aufgrund der geringeren räumlichen Auflösung der Feature-Maps oft nicht detektiert werden.
- **YOLOv3 (Standard)** nutzt das "Darknet-53" Backbone mit 106 Layern. Dies ermöglicht eine deutlich präzisere Extraktion von Merkmalen auch bei schwierigen Lichtverhältnissen oder Teilverdeckungen.

Tabelle 4. Gegenüberstellung der Detektionsmodelle

Merkmal	YOLOv3 (Standard)	YOLOv3-Tiny	Entscheidungsgrundlage
Architektur	106 Layer (Darknet-53)	33 Layer	Standard-Modell bietet tiefere Merkmalsextraktion
Genauigkeit (mAP)	57.9 % (auf COCO)	33.1 % (auf COCO)	Tiny ist für zuverlässige Parkplatzüberwachung zu ungenau
Inferenzzeit	ca. 200 ms (CPU)	ca. 30 ms (CPU)	Latenz des Standard-Modells ist für "Parken" akzeptabel
Kleinerkennung	Hoch	Niedrig	Standard-Modell erkennt auch entfernte Fahrzeuge sicher

Da die Inferenz auf einem Laptop (und nicht auf dem Mikrocontroller selbst) erfolgt, ist die höhere Rechenlast des Standard-Modells akzeptabel, um die geforderte Detektionsgenauigkeit (Mean Average Precision) zu erreichen.

6 Realisierung

6.1 Abstandsrealisierung (Daniel Egbe)

6.1.1 Elektrische Verschaltung und Pin-Belegung

Die physische Anbindung des Ultraschallsensors an den Arduino wird in Tabelle 5 geschildert:

Tabelle 5: HC-SR04 Anschlüsse

HC-SR04 Anschlüsse	Arduino Pins
VCC	5 Volt
GND	GND
TRIG	6
ECHO	7

Der HC-SR04 wird mit einer Betriebsspannung von 5V versorgt. Diese wird durch eine Verbindung vom Breadboard zur 5V-Pin des Arduinos abgegriffen. Ebenso wird der Sensor über einer Verbindung über das Breadboard mit dem GND-Pin des Arduinos verbunden. Der Trigger-Pin (TRIG) ist mit dem analogen Pin 6 des Arduino verbunden. Der Echo-Pin (ECHO) ist mit dem digitalen Pin 7 des Arduino verbunden und dient zur Erfassung der Signallaufzeit.

Die NeoPixel-LED-Leiste mit acht adressierbaren RGB-LEDs ist an den digitalen Pin 8 angeschlossen. Die Spannungsversorgung der LED-Leiste erfolgt ebenfalls über den 5V-Pin des Arduino.

6.1.2 Mechanischer Aufbau

In der Realisierungsphase wurde der Ultraschallsensor mithilfe einer stabilen Halterung (Stift-Konstruktion) am Boden fixiert. Die Montage erfolgte in einer Höhe von ca. 5 cm über der Fahrbahnoberfläche. Diese spezifische Positionierung wurde gewählt, um ein optimales Sichtfeld zu gewährleisten.

Durch den Abstand zum Boden wird verhindert, dass Bodenunebenheiten oder die Fahrbahn selbst ungewollte Echos erzeugen, die als Hindernis missinterpretiert werden könnten.

Die Befestigung an einem Stift ermöglicht zudem eine genaue Ausrichtung des Sensors parallel zur Fahrbahn. Eine Neigung des Sensors nach unten führt zu permanenten Fehlmessungen, während eine Neigung nach oben dazu führt, dass die Hindernisse im Nahbereich nicht erkannt werden.

6.1.3 Implementierung der Messung und Ansteuerlogik

Die Realisierung der Distanzmessung erfolgt innerhalb der Funktion `measureDistance()`. Dabei wird über die entsprechenden Ein- und Ausgänge ein Ultraschallimpuls ausgelöst. Unter Anwendung der in Formel (1) dargestellten Beziehung wird aus der vom Sensor gemessenen Signallaufzeit der Abstand berechnet.

Zunächst wird der Trigger-Pin für $2\mu\text{s}$ auf LOW gesetzt, um einen sauberen Startzustand zu garantieren, gefolgt vom $10\mu\text{s}$ HIGH-Impuls, welcher einen Ultraschallimpuls sendet, der $10\mu\text{s}$ andauert.

Mit der Funktion `pulseln(ECHO_PIN, HIGH)` wird die Dauer, die da Echo-Signals zum Objekt und zurück braucht, in Mikrosekunden erfasst.

In der Formel (1) wird für die Geschwindigkeit die Schallgeschwindigkeit c_{Schall} verwendet, welche 343m/s bzw. 34300cm/s beträgt. Da die vom Sensor erfasste Signallaufzeit in Mikrosekunden angegeben wird, ist eine entsprechende Umrechnung der Schallgeschwindigkeit erforderlich. Daraus ergibt sich der Faktor $0,034\text{ cm}/\mu\text{s}$, der in der Distanzberechnung Anwendung findet.

Die gemessene Laufzeit der Ultraschallwelle umfasst sowohl den Hin- als auch den Rückweg zwischen Sensor und Hindernis. Da für die Berechnung der tatsächlichen Entfernung lediglich der einfache Weg relevant ist, wird die ermittelte Distanz durch zwei dividiert. Dies kann man in Abbildung 1 beobachten.

```
90  ✓ long measureDistance() {  
91      digitalWrite(TRIG_PIN, LOW);  
92      delayMicroseconds(2);  
93      digitalWrite(TRIG_PIN, HIGH);  
94      delayMicroseconds(10);  
95      digitalWrite(TRIG_PIN, LOW);  
96  
97      long duration = pulseIn(ECHO_PIN, HIGH);  
98      long distance = duration * 0.034 / 2;  
99  
100     return distance;  
101 }
```

Abbildung 1: Code zur Distanzberechnung

6.1.4 Datentyp-Wahl

Im vorliegenden Code (Abbildung 2) wird die Variable distance als Datentyp long deklariert. Dies ist eine bewusste Entscheidung, um den Wertebereich gegenüber einem einfachen int zu vergrößern und potenzielle Überläufe bei der Berechnung mit großen Mikrosekunden-Werten der Funktion pulseIn() zu vermeiden.

```
44  ✓ void loop() {  
45      // Ultraschall-Messung (läuft immer)  
46      long distance = measureDistance();  
47      updateLEDs(distance);  
48 }
```

Abbildung 2: Datentyp

6.1.5 Realisierung der visuellen Feedback-Stufen

Die visuelle Rückmeldung erfolgt über eine LED-Leiste mit acht adressbaren LEDs. Die Realisierung im Code (Abbildung 3) nutzt eine if-else Fallunterscheidung innerhalb der Funktion updateLEDs(long distance), um eine Trennung der Warnstufen zu erreichen. Die unterschiedlichen Stufen werden wie folgt unterschieden:

Zone 1 wird durch die Bedingung if (distance > 100) im Code definiert. Detektiert der Ultraschall einen Abstand von mehr als 100 cm, werden mittels des Befehls strip.Color(0, 255, 0) alle acht LEDs auf grün gesetzt. In dieser Zone ist das Kollisionsrisiko minimal. Die grüne Farbgebung signalisiert dem Fahrzeugführer entsprechend einen sicheren Zustand ohne unmittelbare Gefährdung. Durch das gewählte Update-Intervall von 100 ms reagiert die LED-Anzeige unmittelbar, sobald der gemessene Abstand die 100-cm-Schwelle unterschreitet

Zone 2 wird beschrieben durch die Bedingung `else if (distance >= 50)`. Umfasst wird der Distanzbereich von 50 bis 100 cm. Befindet sich das Kraftfahrzeug in diesem Bereich, wechseln die LEDs mithilfe des Befehls `strip.Color(255, 165, 0)` auf Orange. Die orangefarbene Anzeige dient als Hinweis auf eine zunehmende Annäherung an die Parkbegrenzung und erfordert eine erhöhte Aufmerksamkeit des Fahrers. Die gleichzeitige Umschaltung der gesamten LED-Leiste stellt sicher, dass die Warnung im Sichtfeld des Fahrers deutlich wahrgenommen wird.

Die Bedingung `else { color =strip.Color(255, 0, 0);}` beschreibt die 3. Zone. Unterschreitet der gemessene Abstand einen Wert von 50cm, wird die LED-Anzeige auf Rot gesetzt. Dieser Bereich stellt die kritische Zone dar, in der unmittelbare Kollisionsgefahr besteht. Die rote Anzeige bleibt bis zur unteren Messgrenze des Sensors (ca. 2cm) aktiv. Selbst wenn die Messung aufgrund zu geringer Distanz instabil wird, bleibt die Farbe durch die `else`-Bedingung bestehen.

```

103  void updateLEDs(long distance) {
104      uint32_t color;
105
106      if (distance > 100) {          // > 1m = GRÜN
107          color = strip.Color(0, 255, 0);
108      }
109      else if (distance >= 50) {     // 50cm - 1m = ORANGE
110          color = strip.Color(255, 165, 0);
111      }
112      else {                         // < 50cm = ROT
113          color = strip.Color(255, 0, 0);
114      }
115

```

Abbildung 3: Warnstufen

6.1.6 Hardware-Software-Kopplung der Anzeige

Ein entscheidendes Detail der Realisierung ist die `for`-Schleife, die die gewählte Farbe auf alle Pixel überträgt.

Durch das Senden des `strip.show()`-Befehls (Abbildung 4) erst nach der Schleife wird sichergestellt, dass alle LEDs gleichzeitig umschalten. Dies verhindert, dass jede der acht LEDs nacheinander die Farbe wechselt [29].

Die Helligkeit der NeoPixel wird bewusst nicht auf das Maximum gesetzt, um Spannungsabfälle am Arduino zu vermeiden, die wiederum die Genauigkeit des Ultraschallsensors beeinflussen könnten.

```
116      // Alle 8 LEDs auf die Farbe setzen
117      for(int i = 0; i < 8; i++) {
118          strip.setPixelColor(i, color);
119      }
120      strip.show();
```

Abbildung 4: LED einschalten

6.1.7 Verifikation durch Serielle Überwachung

Zur Validierung der Zonenübergänge wurde eine serielle Ausgabe implementiert (Abbildung 5), die alle 2000 ms (lastDistancePrint) den exakten Zentimeterwert mit der aktiven Zone abgleicht.

```
Entfernung: 262 cm - GRÜN
Entfernung: 85 cm - ORANGE
Entfernung: 80 cm - ORANGE
Entfernung: 75 cm - ORANGE
Entfernung: 0 cm - ROT
```

Abbildung 5: Ausgabebeispiel

Dies ermöglichte während der Realisierungsphase eine präzise Feinbestimmung der Schwellwerte, um Messungsungenauigkeiten an den Zonengrenzen zu bewerten.

6.2 Zeitbasierte Helligkeitssteuerung (Serhat Subasi)

Für die Umsetzung der zeitabhängigen Helligkeitssteuerung wird die Echtzeituhr DS1307 eingesetzt. Die Stromversorgung erfolgt über den 5V-Ausgang (VCC) und Masse (GND) des Arduino Microcontrollers. Die Datenkommunikation über das I²C-Protokoll wird durch Verbindung der Leitungen SDA mit Pin A4 und SCL mit Pin A5 des Arduinos hergestellt.

Zur Ansteuerung der Hardware-Komponenten werden die Bibliotheken RTCLib (für die Echtzeituhr) und Adafruit_NeoPixel (für die LED-Ansteuerung) eingebunden. Diese Bibliotheken stellen die notwendigen Klassen und Methoden bereit, um Uhrzeitdaten auszulesen und LEDs hinsichtlich Farbe und Helligkeit anzusteuern.

Die Echtzeituhr wird über folgenden Befehl wie in Listing 1 dargestellt initialisiert.

```
rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
```

Listing 1: Uhrzeit Initialisierung

Dabei wird die Systemzeit der Echtzeituhr auf den Zeitpunkt der letzten Kompilierung gesetzt, sodass die Uhr bei der ersten Inbetriebnahme mit einer korrekten Startzeit arbeitet.

Die aktuelle Uhrzeit wird anschließend aus der Echtzeituhr ausgelesen und in einer Variable gespeichert, wie in Listing 2 dargestellt.

```
DateTime now = rtc.now();  
Int stunde =now.hour();
```

Listing 2: Uhrzeit speichern

Aus dem DateTime-Objekt wird die aktuelle Stunde extrahiert. Diese dient als Grundlage für die zeitabhängige Anpassung der LED-Helligkeit.

Die Adafruit_NeoPixel-Bibliothek stellt mit der Methode setBrightness() eine Möglichkeit zur Verfügung, die Helligkeit aller LEDs im Bereich von 0 (ausgeschaltet) bis 255 (maximale Helligkeit) zu regeln.

Die Helligkeitsanpassung erfolgt in Abhängigkeit von der aktuellen Uhrzeit wie in Listing 3 dargestellt.

```
if (stunde >= 20 || stunde < 8) {    // Nachtbetrieb  
    strip.setBrightness(40);        // reduzierte Helligkeit  
} else {  
    strip.setBrightness(150);       // erhöhte Helligkeit  
}
```

Listing 3: Helligkeitsregelung

In diesem Programmabschnitt wird geprüft, ob sich die aktuelle Uhrzeit im definierten Nachtzeitraum befindet. Dieser umfasst die Zeit ab 20:00 Uhr sowie den Zeitraum

zwischen 00:00 Uhr und 08:00 Uhr. Liegt die aktuelle Stunde innerhalb dieses Bereichs, wird die Helligkeit des LED-Streifens auf einen reduzierten Wert gesetzt. Außerhalb dieses Zeitfensters wird die Helligkeit wieder erhöht.

Auf diese Weise wird sichergestellt, dass die LEDs während der gesamten Nacht gedimmt betrieben werden und tagsüber eine höhere Leuchtstärke aufweisen.

Die **Abbildung 6** und die **Abbildung 7** zeigen die unterschiedlichen Helligkeiten der optischen Rückmeldung. **Tagbetrieb Nachtbetrieb**



Abbildung 6: Tagbetrieb



Abbildung 7: Nachtbetrieb

Wie in den Abbildungen deutlich zu erkennen ist, leuchten die LEDs im Tagesbetrieb mit höherer Helligkeit, während im Nachtbetrieb die Helligkeit reduziert wird. Damit wird die ordnungsgemäße Funktion der zeitabhängigen Helligkeitsregelung bestätigt.

6.3 Displayausgabe (Serhat Subasi)

Für die Displayrückmeldung wird das LCD1602-Modul mit dem I²C-Adapter eingesetzt, das über einen Pinboard mit dem Arduino verbunden wird. Die Stromversorgung erfolgt über den VCC-Anschluss des Adapters an den 5V-Pin des Arduinos sowie über GND. Die Datenkommunikation über das I²C-Protokoll wird durch die Verbindung der SDA-Leitung mit Pin A4 und der SCL-Leitung mit Pin A5 des Arduinos umgesetzt.

Für die Implementierung wurden die Bibliotheken RTCLib und LiquidCrystal_I2C eingebunden. Die RTCLib-Bibliothek wird verwendet, um Ein- und Auscheck-Uhrzeiten zu erfassen und zu speichern. Die LiquidCrystal_I2C-Bibliothek stellt Funktionen zur Verfügung, welche die Steuerung des Displays ermöglichen.

Zur Steuerung des Systemzustands wird eine boolesche Variable mit dem Namen Status, welche auf false initialisiert ist verwendet. Diese Variable bestimmt, ob sich das System im Eincheck- oder Auscheck-Modus befindet.

Nach dem erfolgreichen Einlesen einer RFID-Karte wird der aktuelle Systemstatus umgeschaltet- Durch die Invertierung der status-Variable wird bei jedem Karten-Scan zwischen Einchecken und Auschecken gewechselt.

Listing 4 zeigt den Befehl für die Umschaltung.

```
status = !status;
```

Listing 4: Status umschalten

Die Variable wird ausschließlich bei einem Ein- oder Auscheck-Vorgang per RFID-Karte invertiert. Anschließend wird sie als Parameter an die Funktion display() übergeben, welche für die Ausgabe der Informationen auf dem Display verantwortlich ist.

Bei einem Eincheck-Vorgang wird sie auf true gesetzt, wodurch die in Listing 5 gezeigte Fallunterscheidung ausgelöst wird.

```
if (status) {  
    // Einchecken  
    checkInTime = rtc.now();  
    lcd.setCursor(0, 0);    // setzt den Cursor auf die erste Zeile und  
                           // erste Spalte  
    lcd.print("Es wurde");  // gibt den Text auf dem Display aus  
    lcd.setCursor(0, 1);    // setzt den Cursor auf die zweite Zeile  
    lcd.print("Eingecheckt!");  
    delay(5000);  
    lcd.clear();  
}
```

Listing 5: Eincheck Rückmeldung

Dabei wird die aktuelle Uhrzeit mithilfe der Echtzeituhr in der Variable checkInTime gespeichert. Anschließend wird auf dem Display eine Bestätigung des Eincheck-Vorgangs ausgegeben. Durch den Befehl delay(5000) bleibt die Anzeige für fünf Sekunden sichtbar, bevor das Display mit lcd.clear() geleert wird.

Bei einem Auscheck-Vorgang setzt das System die Variable status auf false und führt den else-Zweig der display()-Funktion nach der in Listing 6 gezeigten Logik aus.

```
else {  
    // Auschecken  
    DateTime checkOutTime = rtc.now();    // holt die aktuelle Zeit  
    TimeSpan parkdauer = checkOutTime - checkInTime; // berechnet die  
                                                    // Parkdauer durch die Differenz  
    ...
```

Listing 6: Speicherung der Checkout-Zeit

Die Auscheckuhrzeit in der Variable `checkOutTime` gespeichert. Daraus wird die Parkdauer als Differenz zum Eincheck-Zeitpunkt berechnet und in der Variablen `parkdauer` abgespeichert. Anschließend erscheint für drei Sekunden eine Bestätigung des Auscheckvorgangs auf dem Display. Listing 7 zeigt den Code für die Ausgabe der Parkdauer.

```
lcd.setCursor(0, 0);  
lcd.print("Parkdauer:");  
lcd.setCursor(0, 1);  
lcd.print(parkdauer.hours());    // gibt die Stunde aus  
lcd.print("h ");  
lcd.print(parkdauer.minutes()); // gibt die Minute aus  
lcd.print("m ");  
lcd.print(parkdauer.seconds()); // gibt die Sekunde aus  
lcd.print("s");
```

Listing 7: Uhrzeit speichern

Hier wird der Cursor an die obere linke Ecke gesetzt und die Parkdauer in Stunden, Minuten und Sekunden ausgegeben. Nach drei Sekunden wird die Anzeige gelöscht und das System auf den nächsten Check-In vorbereitet.

In Abbildung 8 zeigt die Anzeige auf dem Display nach einem Auscheck-Vorgang die Parkdauer.



Abbildung 8: Anzeige der Parkdauer

Die Abbildung 8 belegt die korrekte Anzeige der Systemrückmeldung und der Parkdauer. Somit ist die Umsetzung dieser Funktion vollständig realisiert.

6.4 MFRC522 und Arduino-Software (Vincent Gentz)

Das MFRC522 RFID-Modul wird zur Verifikation des Nutzers verwendet. Die folgende Tabelle 6 zeigt die verwendete Pin-Belegung:

Tabelle 6: MFRC522-Anschlüsse

Arduino-Anschlüsse	MFRC522-Anschlüsse
VCC	3,3 Volt
GND	GND
RST	9
SDA	10
MOSI	11
MISO	12
SCK	13

Der MFRC522 wird über die Pins VCC und GND mit dem Arduino verbunden. Für alle übrigen Anschlüsse werden digitale Pins des Arduinos verwendet, um eine erfolgreiche Steuerfunktion und Datenübertragung sicherzustellen.

Auf Softwareebene des Arduino-Mikrocontrollers wird die MFRC522-Bibliothek verwendet. Diese initialisiert, unterstützt und stellt sämtliche Funktionen für die Steuerung des Lesegeräts bereit. Der Mikrocontroller fragt periodisch ab, um eine erfolgreiche Erkennung eines RFID-Transponders zu garantieren.

Mit Hilfe der `PICC_IsNewCardPresent()` und `PICC_ReadCardSerial()` Funktionen wird ein Lesevorgang ausgelöst. Nach erfolgreichen Lesen wird mit folgenden Anweisungen das Signalwort mit entsprechender Information im Terminal ausgegeben wie es in Listing 8.

```
Serial.print(F("Karte erkannt! UID: "));  
Serial.println(uid);
```

Listing 8: Arduino-Terminal-Ausgabe

Um mehrfaches, unerwünschtes Lesen derselben UID zu verhindern, wurde ein zeitbasiertes Cooldown-Verfahren implementiert. Die Sperrzeit beträgt 15 Sekunden, während dieser Sperrzeit werden sämtliche Erkennungen eines RFID-Transponders ignoriert. Die Logik vergleicht hierbei die aktuelle Systemzeit (`millis()`) mit dem Zeitpunkt der letzten erfolgreich gescannten UID. Umgesetzt wurde es folgendermaßen in Listing 9.

```
unsigned long timePassed = millis() - cooldownStart;  
unsigned long remaining = cooldownTime - timePassed;  
  
// Logik verbleibende Zeit  
if (timePassed >= cooldownTime) {  
    cooldownActive = false;  
    //Cooldown beendet  
}  
  
return;
```

Listing 9: Cooldown

Die Variable `timePassed` repräsentiert die verstrichene Zeit, seit der letzten erfolgreich gescannten UID. Die verbleibende Sperrzeit wird in der Variable `remaining`

gespeichert und wird für die Anzeige am LCD verwendet. Sobald der Cooldown vorbei ist, wird `cooldownActive` auf `false` gesetzt, sodass der Mikrocontroller wieder gültige Erkennungen verarbeitet.

6.5 Python Datenbank und GUI (Vincent Gentz)

Das erste Python-Modul umfasst die Datenbankinitialisierung und die dazugehörigen Methoden, während der zweite Modulteil das GUI (Graphical User Interface) umfasst. Die Datenbank dient zur Verwaltung aller relevanten Informationen. Die zentrale Tabelle beinhaltet die Parameter `id`, `vorname`, `nachname`, `parkplatznummer`, `belegt` und `eingecheckt`. Der Parameter `belegt` wird für die Nutzung des Parkplatzes verwendet, während `eingecheckt` für die Verifizierung des Fahrers genutzt wird.

```
cur.execute('''
    CREATE TABLE IF NOT EXISTS personen (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        vorname TEXT NOT NULL,
        nachname TEXT NOT NULL,
        parkplatznummer TEXT NOT NULL,
        belegt INTEGER DEFAULT 0,
        eingecheckt INTEGER DEFAULT 0
    )
```

Listing 10: Datenbank-Tabellen-Struktur

Die Initialisierung und Erstellung erfolgten über in Listing 10 dargestellten Code. Die Spalte `id` dient als eindeutiger Primärschlüssel und wird durch die UUIDs der jeweiligen Chips definiert.

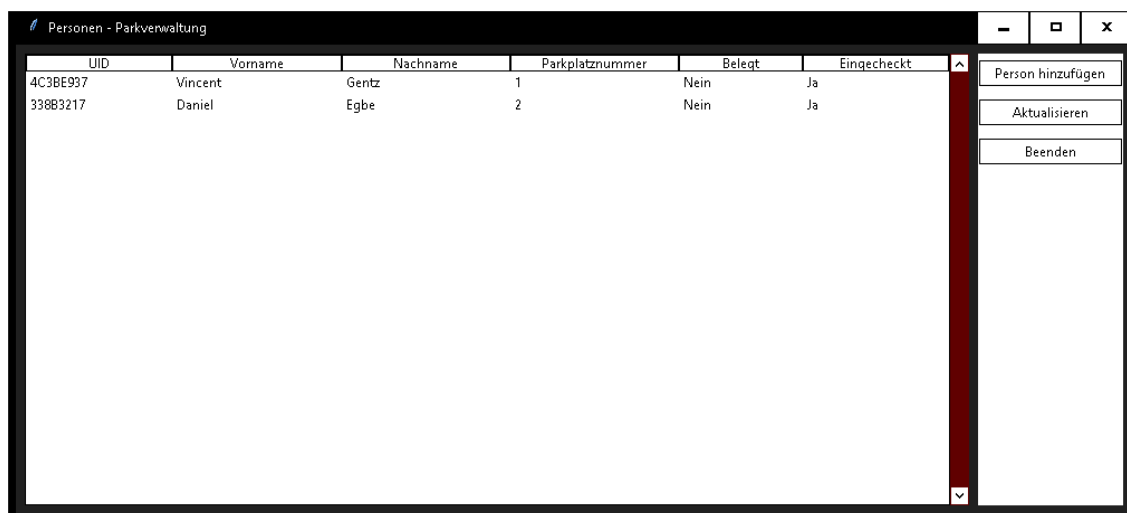
Zur Verwaltung der Datenbank wurden mehrere Funktionen implementiert, die in Listing 11 dargestellt sind.

```
def add_person(vorname: str, nachname: str, parkplatznummer: str, path:
Optional[str] = None) -> int:
def delete_person(vorname: str, nachname: str, parkplatznummer: str, path:
Optional[str] = None) -> int:
def show_all_personen():
def update_eingecheckt(vorname: str, nachname: str, eingecheckt: int, path:
Optional[str] = None) -> int:
def update_belegt(vorname: str, nachname: str, belegt: int, path:
Optional[str] = None) -> int:
def check(parkplatznummer: str):
```

Listing 11: Tabellen-Funktionen

Die Funktion `add_person()` fügt der Tabelle eine neue Person hinzu. Mit Hilfe `delete_person()` kann diese wieder entfernt werden. Mit `show_all_personen()` lässt sich der vollständige Inhalt der Tabelle anzeigen, diese Funktion dient für den Überblick aller registrierter Nutzer und deren personenbezogenen Daten. Für die Manipulation stehen die Funktionen `update_eingecheckt()` und `update_belegt()` zur Verfügung, mit Hilfe dieser können die Werte in der Tabelle zur jeweils zugehörigen id verändert werden. Da im Projekt von einem privaten Parkplatz ausgegangen wird, wird eine weitere Funktion `check()` benötigt. Diese Funktion vergleicht die belegt- und eingecheckt-Werte, um nicht autorisierte Parkvorgänge zu identifizieren.

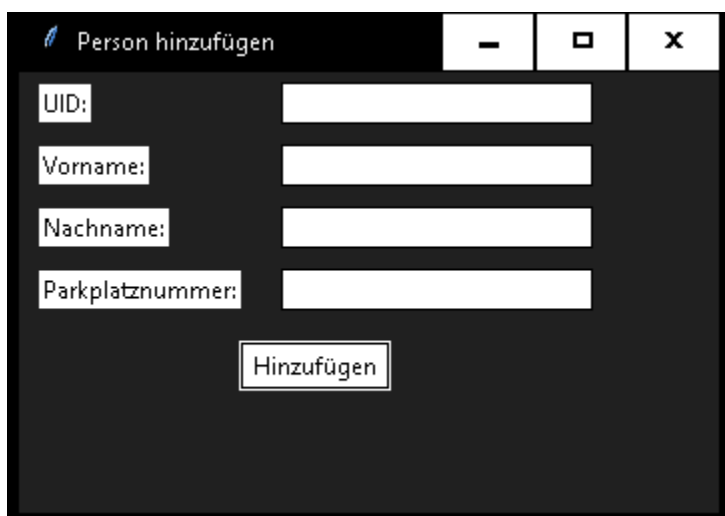
Zur Interaktion mit der Datenbank wurde eine GUI implementiert, die Einsicht und Manipulation mit Hilfe der oben definierten Funktionen erlaubt. Die folgende **Abbildung 9** zeigt die GUI der Personen- und Parkverwaltung.



UID	Vorname	Nachname	Parkplatznummer	Belegt	Eingeecheckt
4C3BE937	Vincent	Gentz	1	Nein	Ja
338B3217	Daniel	Egbe	2	Nein	Ja

Abbildung 9: GUI-Parkverwaltung

Über die GUI können Personen zudem hinzugefügt werden, hierzu dient der “Person hinzufügen” Button. Falls man diesen betätigt, öffnet sich wie in **Abbildung 10** ein weiteres Fenster. Hier wird die UID, der Vorname, Nachname und Parkplatznummer benötigt.



Person hinzufügen

UID:

Vorname:

Nachname:

Parkplatznummer:

Abbildung 10: Person-hinzufügen-GUI

Nach Eingabe der relevanten Informationen, werden die Information für das Anlegen einer neuen Person verwendet. Diese kann in den Personen – Parkverwaltung über einen Rechtsklick wieder entfernt werden wie in **Abbildung 11** zu sehen ist. Zur zusätzlichen Sicherheitsüberprüfung wird ein Bestätigungsfenster angezeigt.

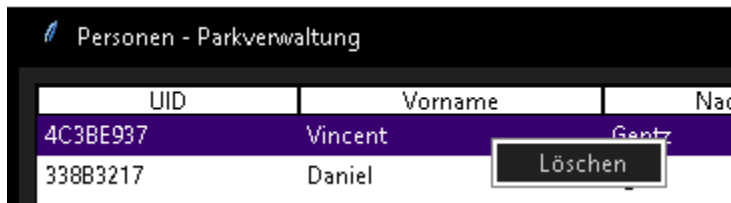


Abbildung 11: Löschen-GUI

Bei allen Interaktionen im GUI werden die entsprechenden Funktionen der Datenbank zur Verarbeitung der Daten aufgerufen.

6.6 Python Schnittstelle (Vincent Gentz)

Die Schnittstelle zwischen dem Arduino-System und dem Datenbank-System wird mithilfe einer Polling-Architektur implementiert. Als praktische Methodik wird ein Listener eingesetzt, der die serielle Ausgabe des Arduino-Systems aktiv abhört.

Das Auslesen der Daten erfolgt zeilenweise, wobei das System definierte Signalwörter abfängt. Die Überwachung der seriellen Ausgabe ist ereignisgesteuert und auf spezifische Signalwörter ausgerichtet wie „UID:“ und „Parkplatz:“

Sobald ein Signalwort erkannt wird, decodiert der Listener die empfangene Zeile. Die Umsetzung dieser Dekodierung ist in Listing 12 dargestellt.

```
line = ser.readline().decode('utf-8', errors='ignore').strip()
```

Listing 12: Zeilen-Decodierung

Im Anschluss an die Dekodierung erfolgt die Datenextraktion durch eine String-Analyse. Diese extrahiert Signalwort und dazugehörige Information und speichert die Informationen in vorgesehene Variablen. Durch diese Zwischenspeicherung und Extraktion lässt sich die entsprechende Funktion der Datenbanklogik direkt aufrufen und anwenden.

6.7 Implementierung der Firmware (ESP32) (Tom Liebegall)

Die Firmware wurde in der Arduino-IDE entwickelt und basiert auf der esp32cam-Bibliothek. Zunächst wird ein Überblick über die Konfiguration der Kamera-Hardware gegeben. Kennzeichnend für den gewählten Ansatz ist die Optimierung des Speichermanagements, um trotz der begrenzten Ressourcen des Mikrocontrollers eine stabile Bildübertragung zu gewährleisten.

Initialisierung und Puffer-Konfiguration

Wie im Listing 13 zu sehen ist, wird im `setup()` die Konfiguration festgelegt:

```
using namespace esp32cam;
Config cfg;
cfg.setPins(pins::AiThinker);
cfg.setResolution(hiRes);
cfg.setBufferCount(2);
cfg.setJpeg(80);
```

Listing 13: Konfiguration des Kameraspeichers

Hier (Listing 13) ist besonders die Zeile `cfg.setBufferCount(2)` hervorzuheben. Durch diese Einstellung wird ein sogenanntes Double-Buffering aktiviert. Dies hat den Vorteil, dass der DMA-Controller (Direct Memory Access) bereits das nächste Bild in den Speicher schreiben kann, während das vorherige Bild noch über das Netzwerk gesendet wird. Aus diesem Grund können Latenzen minimiert und die Framerate stabilisiert werden.

Webserver und Routing

Die Bereitstellung der Bilder erfolgt über einen HTTP-Server auf Port 80. Listing 14 definiert hierfür spezifische Routen (Endpunkte):

```
server.on("/cam-lo.jpg", handleJpgLo);
server.on("/cam-hi.jpg", handleJpgHi);
server.on("/cam-mid.jpg", handleJpgMid);
```

Listing 14: Kamera Auflösung

Die Funktion `handleJpgLo` ändert dynamisch die Auflösung auf 320x240 Pixel (loRes), bevor das Bild gesendet wird. Dies ermöglicht es dem Client, je nach

Netzwerkauslastung flexibel zwischen hoher Qualität und schneller Übertragung zu wechseln, ohne dass der Controller neu gestartet werden muss.

6.7.1 Bildverarbeitung und Objekterkennung (Python)

Der zweite Teil der Realisierung widmet sich der clientseitigen Verarbeitung. Das Python-Skript nutzt die Bibliotheken OpenCV (cv2) für die Bildmanipulation und NumPy für mathematische Operationen.

Bildakquise und Vorverarbeitung

Zunächst muss das Bild vom ESP32 geladen werden. Da OpenCV keine direkte URL-Unterstützung für Einzelbild-Abrufe in diesem Format bietet, wird die Bibliothek `urllib.request` genutzt:

```
img_resp = urllib.request.urlopen(url)
imgnp = np.array(bytearray(img_resp.read()), dtype=np.uint8)
im = cv2.imdecode(imgnp, -1)
```

Listing 15: Abruf und Dekodierung des Bilddatenstroms via HTTP

Hier (Listing 15) wird der rohe Byte-Stream (`img_resp.read()`) in ein eindimensionales NumPy-Array (`uint8`) konvertiert. Anschließend dekodiert `cv2.imdecode` dieses Array in ein interpretierbares Bildformat (BGR-Matrix).

Vorbereitung für das neuronale Netz (Blobbing)

Bevor das Bild an das YOLO-Netzwerk übergeben werden kann, muss es in ein spezifisches Format, den sogenannten Blob, transformiert werden. Kennzeichnend dafür ist folgender Code-Abschnitt Listing 16:

```
blob = cv2.dnn.blobFromImage(im, 1/255, (whT, whT), [0, 0, 0], 1,
crop=False)
net.setInput(blob)
```

Listing 16: Vorverarbeitung und Normalisierung für das neuronale Netz

Dabei passieren drei Dinge:

1. **Skalierung:** Die Pixelwerte werden durch den Faktor $1/255$ auf einen Bereich zwischen 0 und 1 normalisiert.
2. **Resizing:** Das Bild wird auf die Dimension (320, 320) (definiert als whT) skaliert, da das trainierte YOLO-Modell diese Eingangsgröße erwartet.
3. **Input:** Der Blob wird als Input für das Netzwerk (net) gesetzt.

Inferenz und Filterung (Non-Maximum Suppression)

Nach dem Forward-Pass (net.forward) liefert das Netz eine Vielzahl von Bounding Boxes zurück. Problematisch scheint vor allem, dass für ein einziges Objekt oft mehrere überlappende Boxen erkannt werden. Um dies zu lösen, wurde die Non-Maximum Suppression (NMS) implementiert (Listing 17):

```
indices = cv2.dnn.NMSBoxes(bbox, confs, confThreshold, nmsThreshold)
```

Listing 17: Filterung redundanter Detektionen mittels Non-Maximum Suppression

Diese Funktion führt eine mathematische Filterung durch. Sie berechnet die *Intersection over Union* (IoU) der Boxen. Wenn zwei Boxen zu stark überlappen (hier definiert durch `nmsThreshold = 0.3`) und dieselbe Klasse beschreiben, wird nur diejenige mit der höchsten Wahrscheinlichkeit (`confs`) behalten. Daraus ergibt sich, dass jedes Fahrzeug im Bild eindeutig nur einmal markiert wird.

6.7.2 Physischer Aufbau und Hardware-Optimierung

Ein wesentlicher Aspekt der Realisierung war die Sicherstellung einer stabilen Datenübertragung. Bei der Betrachtung der WLAN-Leistung des ESP32-CAM musste berücksichtigt werden, dass die integrierte PCB-Antenne eine nur geringe Reichweite besitzt.

Aus diesem Grund wurde eine hardwareseitige Anpassung vorgenommen. Der 0-Ohm-Widerstand auf der Platine, der den Signalpfad zur internen Antenne brückt, wurde umgelötet, um den externen IPEX-Konnektor zu aktivieren. Dies ermöglicht den Anschluss einer leistungsfähigeren 2,4 GHz Stabantenne. Tests zeigten, dass diese Maßnahme die Verbindungsstabilität signifikant erhöhte.

Für die finale Montage ist ein Gehäuse vorgesehen, welches mittels 3D-Druck gefertigt wird, um die Elektronik zu schützen. Zusammenfassend lässt sich sagen,

dass durch die Kombination aus optimierter Firmware, effizienter Python-Verarbeitung und Hardware-Anpassungen ein funktionsfähiges Echtzeitsystem realisiert werden konnte.

7 Zusammenfassung und Ausblick (Tom Liebegall)

7.1 Zusammenfassung

Im Rahmen dieser Projektarbeit wurde ein externes Parkassistenzsystem entwickelt, das darauf abzielt, den Parkvorgang durch den Einsatz kostengünstiger IoT-Hardware zu erleichtern und sicherer zu gestalten. Der Fokus lag dabei auf der Kombination zweier unterschiedlicher Sensor-Technologien: einer optischen Überwachung mittels ESP32-CAM und einer präzisen Distanzmessung durch Ultraschallsensoren.

Für die optische Komponente wurde ein Client-Server-System implementiert. Der Mikrocontroller fungiert hierbei als Webserver, der kontinuierlich Bilder der Parksituation bereitstellt. Diese werden von einem externen Rechner abgerufen und mithilfe des YOLOv3-Algorithmus analysiert. Es konnte gezeigt werden, dass dieses Verfahren Fahrzeuge zuverlässig erkennt und klassifiziert, was eine digitale Belegungserkennung ermöglicht. Um die Reichweitenprobleme der Standard-Hardware zu umgehen, wurde das System erfolgreich durch eine externe Antenne optimiert.

Parallel dazu wurde ein Ultraschall-basiertes Warnsystem realisiert, das den Abstand zum Fahrzeug misst und dem Fahrer über eine LED-Leiste visuelles Feedback gibt. Durch die Einteilung in drei Farbzonen (Grün, Orange, Rot) erhält der Nutzer eine intuitive Rückmeldung, wie weit er noch vorfahren kann. Die Tests bestätigten, dass diese Kombination aus Sensorik und visueller Ausgabe präzise arbeitet und Hindernisse im Nahbereich zuverlässig erkennt. Beide Teilsysteme demonstrieren, dass sich mit verhältnismäßig geringem Hardware-Aufwand effiziente Lösungen für die Parkraumüberwachung realisieren lassen.

7.2 Ausblick

Obwohl der entwickelte Prototyp die grundlegenden Anforderungen erfüllt, zeigten sich im Projektverlauf Aspekte, die in zukünftigen Entwicklungsstufen verbessert werden könnten. Ein wesentlicher Punkt ist die Abhängigkeit des Kamerasystems von guten Lichtverhältnissen. Bei Dunkelheit sinkt die Erkennungsrate deutlich, weshalb eine Erweiterung um Infrarot-Scheinwerfer oder der Einsatz lichtempfindlicherer Sensoren sinnvoll wäre. Auch die Latenz bei der Bildübertragung, verursacht durch

das WLAN-Netzwerk, könnte durch leistungsfähigere Hardware oder eine direkte Verarbeitung auf dem Chip (Edge Computing) reduziert werden.

Für das Ultraschallsystem wäre eine drahtlose Anbindung an eine zentrale Steuereinheit denkbar, um Kabelsalat zu vermeiden und die Installation flexibler zu gestalten. Zudem bietet das gesammelte Datenmaterial Potenzial für weitere Funktionen: So könnte das System in Zukunft nicht nur anzeigen, ob ein Parkplatz frei ist, sondern über eine App auch Statistiken zur Auslastung liefern oder mittels Nummernschilderkennung eine Schrankensteuerung automatisieren. Das Projekt bildet somit eine solide Basis für ein umfassendes Smart-Parking-System.

Literaturverzeichnis

- [1] J. S. Hunter, „Buisnessinsider.de,“ Axel Springer Deutschland GmbH, 08 11 2017. [Online]. Available: <https://www.businessinsider.de/tech/autos-werden-seit-jahrzehnten-immer-breiter-strassen-nicht-2017-11/>. [Zugriff am 13 12 2025].
- [2] J. Vargas, S. Alsweiss, O. Toker, R. Razdan und J. Santos, „An Overview of Autonomous Vehicles Sensors and Their Vulnerability to Weather Conditions,“ *Sensors*, Bd. 21, Nr. 16, p. 5397, 2021.
- [3] C. Technologies, „cdm.awsli.com,“ 2013. [Online]. Available: https://docs.google.com/document/d/1Y-yZnNhMYy7rwhAgyL_pfa39RsB-x2qR4vP8saG73rE/edit. [Zugriff am 19 12 2025].
- [4] C. W. de Silva, *Sensor Systems: Fundamentals and Applications*, Boca Raton: CRC Press, 2016.
- [5] A. Rahman, „Precision and Accuracy of Ultrasonic and Infrared Laser ToF IoT Sensors,“ *JOURNAL OF INFORMATICS AND TELECOMMUNIKATION ENGINEERING*, Bd. 8, Nr. 2, 21 1 2025.
- [6] A. Device, „DS1307 64×8, Serial, I²C Real-Time Clock (RTC) Datasheet. <https://www.analog.com/media/en/technical-documentation/data-sheets/ds1307.pdf> Zugriff: 24.11.2025“.
- [7] C. Kühnel, „Hard- und Software Open Source Plattform. Verfügbar: <https://books.google.de/books?id=7DcGkj7O9nIC> : 24.11.2025,“ in *Arduino*, Skript Verlag Kühnel, 2011, p. 67.
- [8] M. Hossain, „Design and study of silicon microring resonator based all-optical binary-coded decimal adder, *Optical and Quantum Electronics*,“ p. 8, 28 September 2023.
- [9] S. Labs, „Benchmarks: Real-Time Clocks – DS3231, PCF8563, MCP79400, DS1307 <https://www.switchdoc.com/2014/12/benchmarks-realtime-clocks-ds3231-pcf8563-mcp79400-ds1307/> Zugriff: 24.11.2025,“ 2014.
- [10] I. Vishay, „LCD016N002BCF-HET 16×2 Character LCD Module Datasheet <https://www.vishay.com/docs/37484/lcd016n002bcfhet.pdf> Zugriff: 24.11.2025,“ Vishay Intertechnology, 2013.

-
- [11] H. Ltd, „HD44780U (LCD-II) Dot-Matrix Liquid Crystal Display Controller/Driver Datasheet <https://cdn.sparkfun.com/assets/9/5/f/7/b/HD44780.pdf> Zugriff: 24.11.2025,“ 1999.
- [12] H. Technology, „I2C Serial Interface 1602 LCD Module Datasheet https://www.handsontec.com/dataspecs/module/I2C_1602_LCD.pdf Zugriff: 25.11.2025“.
- [13] M. Lampe, C. Flörkemeier und S. Haller, „Einführung in die RFID-Technologie,“ in *Das Internet der Dinge*, Berlin, Heidelberg, Springer, 2005, pp. 69-86.
- [14] C. Gomez, J. Oller und J. Paradells, „Overview and Evaluation of Bluetooth Low Energy: An Emerging Low-Power Wireless Technology,“ *Sensors*, Bd. 12, Nr. 9, pp. 11734-11753, 2012.
- [15] Y. Shafranovich, „Common Format and MIME Type for Comma-Separated Values (CSV) Files,“ The Internet Society, 2005.
- [16] T. Bray, „The JavaScript Object Notation (JSON) Data Interchange Format,“ IETF, 2017.
- [17] B. Miller, D. Ranum und L. College, „Analysis of Sequential Search,“ in *Problem Solving with Algorithms and Data Structures using Python*, 2014.
- [18] T. P. G. D. Group, „PostgreSQL 18 Documentation,“ 2025. [Online]. Available: <https://www.postgresql.org/docs/18/index.html>. [Zugriff am 13 12 2025].
- [19] R. Bayer und M. E., „Origanization and maintenance of large ordered indices,“ in *Proceedings of the 1970 ACM SIGFIDET (now SIGMOD) Workshop on Data Description, Access and Control*, New York, NY, USA, 1970.
- [20] D. Golec, I. Strugar und D. Belak, „The Benefits of Enterprise Data Warehouse Implementation in Cloud vs. On-premises,“ *ENTRENOVA – ENTERprise REsearch InNOVation Journal*, Bd. 7, Nr. 1, pp. 66-74, 2021.
- [21] J. Yang, D. B. Minturn und F. Hady, „When Poll Is Better than Interrupt,“ San Jose, CA, USA, 2012.
- [22] T. Lin, H. Rivano und F. Le Mouël, „A Survey of Smart Parking Solutions,“ *IEEE Transactions on Intelligent Transportation Systems*, Bd. 18, Nr. 12, p. 3229–3253, 2017.
-

- [23] V. V. S. S. B. Chakravarthi, „A Survey on Smart Parking Systems,“ in *2019 3rd International Conference on Trends in Electronics and Informatics (ICOEI)*, Tirunelveli, India, 2019.
 - [24] G. Amato, F. Carrara, F. Falchi, C. Gennaro und C. Vartico, „Deep learning for smart parking: a comparative study,“ in *2016 IEEE International Symposium on Systems Engineering (ISSE)*, Edinburgh, UK, 2016.
 - [25] J. Redmon und A. Farhadi, „YOLOv3: An Incremental Improvement,“ 2018.
 - [26] W. Yu, F. Liang, X. He und W. G. Hatcher, „A Survey on the Edge Computing for the Internet of Things,“ *IEEE Access*, Bd. 6, p. 6900–6919, 2018.
 - [27] „ESP32-CAM Development Board Datasheet,“ Espressif Systems, 2019. [Online]. Available: <https://www.espressif.com/en/support/documents/technical-documents>.
 - [28] d'Orgeville Ghislain, „digipart.com,“ 2021. [Online]. Available: <https://datasheets.maximintegrated.com/en/ds/MAX7219-MAX7221.pdf>. [Zugriff am 19 12 2025].
 - [29] P. Burgess, „learn.adafruit.com,“ 30 8 2013. [Online]. Available: <https://learn.adafruit.com/adafruit-neopixel-uberguide>. [Zugriff am 19 12 2025].
-

Abbildungsverzeichnis

Abbildung 1: Code zur Distanzberechnung	27
Abbildung 2: Datentyp	27
Abbildung 3: Warnstufen	28
Abbildung 4: LED einschalten	29
Abbildung 5: Ausgabebeispiel	29
Abbildung 6: Tagbetrieb	31
Abbildung 7: Nachtbetrieb.....	31
Abbildung 8: Anzeige der Parkdauer	35
Abbildung 9: GUI-Parkverwaltung	39
Abbildung 10: Person-hinzufügen-GUI	39
Abbildung 11: Löschen-GUI	40

Tabellenverzeichnis

Tabelle 1: Vergleich Sensoren	7
Tabelle 2: Abstand-Farbcodierung	15
Tabelle 3: Technischer Vergleich der Hardware-Optionen	23
Tabelle 4: Gegenüberstellung der Detektionsmodelle	24
Tabelle 5: HC-SR04 Anschlüsse	25
Tabelle 6: MFRC522-Anschlüsse	35

Listingverzeichnis

Listing 1: Uhrzeit Initialisierung	30
Listing 2: Uhrzeit speichern	30
Listing 3: Helligkeitsregelung.....	30
Listing 4: Status umschalten.....	32
Listing 5: Eincheck Rückmeldung.....	33
Listing 6: Speicherung der Checkout-Zeit.....	33
Listing 7: Uhrzeit speichern	34
Listing 8: Arduino-Terminal-Ausgabe.....	36
Listing 9: Cooldown.....	36
Listing 10: Datenbank-Tabellen-Struktur.....	37
Listing 11: Tabellen-Funktionen	38
Listing 12: Zeilen-Decodierung	40
Listing 13: Konfiguration des Kameraspeichers.....	41
Listing 14: Kamera Auflösung	41
Listing 15: Abruf und Dekodierung des Bilddatenstroms via HTTP	42
Listing 16: Vorverarbeitung und Normalisierung für das neuronale Netz	42
Listing 17: Filterung redundanter Detektionen mittels Non-Maximum Suppression	43

Anhang A Python-Code

A.1 Datei: database.py

Beschreibung: Enthält die Initialisierung der Datenbank, Funktionen zur Manipulation von Einträgen und Abfrage von Informationen.

```
import sqlite3
from typing import Optional, List, Dict
# Projektlogik, Methodenplanung und GUI-/Datenbankstruktur stammen vollständig
# vom Autor;
# GitHub Copilot / ChatGPT wurde nur teils für Python-Syntax und
# Bibliotheksdetails genutzt.

"""
database.py
Datenbankfunktionen für Parkplatzverwaltungssystem
@author: Vincent Gentz
Matrikelnummer: 331843
Datum: 22.10.2025"""

DB_PATH = 'database.db'
# Initialisierung der Datenbank und Tabelle 'Personen' falls nicht vorhanden
def initialise_db(path: str = DB_PATH):
    with sqlite3.connect(path) as conn:
        cur = conn.cursor()
        # Tabelle wird mit SQL erstellt
        cur.execute('''
            CREATE TABLE IF NOT EXISTS personen (
                uid TEXT PRIMARY KEY,
                vorname TEXT NOT NULL,
                nachname TEXT NOT NULL,
                parkplatznummer TEXT NOT NULL,
                belegt INTEGER DEFAULT 0,
                eingecheckt INTEGER DEFAULT 0
            )
        ''')
        conn.commit()
        print("Datenbank initialisiert.")
    # Datenbank wird initialisiert, erstellt Tabelle falls nicht vorhanden
    initialise_db()

"""add_person(...) fügt neue Person der Datenbank hinzu
uid: Chip UID für Individuum
```

```

Vor-/Nachname: Name der Person
parkplatznummer: Zugewiesene Parkplatznummer"""

def add_person(uid: str, vorname: str, nachname: str, parkplatznummer: str,
path: Optional[str] = None) -> str:
    # Datenbank wird als Variable geöffnet
    with sqlite3.connect(DB_PATH) as conn:
        cur = conn.cursor() # für SQL Befehle
        try:
            # Einfügen der Person
            cur.execute('INSERT INTO personen (uid, vorname, nachname,
parkplatznummer) VALUES (?, ?, ?, ?)',
                        (uid, vorname, nachname, parkplatznummer))
            print(f"Person {vorname} {nachname} mit UID {uid} wurde
hinzugefügt.")
            return uid
        except sqlite3.Error as e:
            print(f"Fehler beim Hinzufügen der Person: {e}")
            return ""

"""delete_person(...) löscht eine Person aus der Datenbank anhand der UID
uid: Chip UID der zu löschenden Person"""

def delete_person(uid: str, path: Optional[str] = None) -> int:
    # Datenbank wird als Variable geöffnet
    with sqlite3.connect(DB_PATH) as conn:
        cur = conn.cursor() # für SQL Befehle
        try:
            # Löschen der Person
            cur.execute('DELETE FROM personen WHERE uid = ?', (uid,))
            print(f"Person mit UID {uid} wurde gelöscht.")
            return cur.rowcount
        except sqlite3.Error as e:
            print(f"Fehler beim Löschen der Person: {e}")
            return 0

"""show_all_personen() gibt alle Personen in der Datenbank zurück."""

def show_all_personen() -> List[Dict]:
    # Datenbank wird als Variable geöffnet
    with sqlite3.connect(DB_PATH) as conn:
        cur = conn.cursor() # für SQL Befehle
        cur.execute('SELECT uid, vorname, nachname, parkplatznummer, belegt,
eingescheckt FROM personen') # Alle Personen abfragen
        rows = cur.fetchall()
        # Ergebnis in Liste von Dictionaries umwandeln

```

```

        return [
            {
                'uid': row[0],
                'vorname': row[1],
                'nachname': row[2],
                'parkplatznummer': row[3],
                'belegt': row[4],
                'eingecheckt': row[5]
            }
        ]
    for row in rows
]

"""update_eingecheckt(...) ändert eingecheckt-Status einer Person anhand der
UID"""

def update_eingecheckt(uid: str, path: Optional[str] = None) -> int:
    # Datenbank wird als Variable geöffnet
    with sqlite3.connect(DB_PATH) as conn:
        cur = conn.cursor() # für SQL Befehle
        try:
            # Aktuellen Status abfragen
            cur.execute('SELECT eingecheckt FROM personen WHERE uid = ?',
(uid,))
            result = cur.fetchone()

            if result is None:
                print(f"UID {uid} nicht in Datenbank gefunden.")
                return 0

            # Status umkehren (0 -> 1 oder 1 -> 0)
            neuer_status = 1 if result[0] == 0 else 0

            # Status aktualisieren
            cur.execute('UPDATE personen SET eingecheckt = ? WHERE uid = ?',
(neuer_status, uid))
            return cur.rowcount

        except sqlite3.Error as e:
            print(f"Fehler beim Aktualisieren von eingecheckt: {e}")
            return 0

"""update_belegt(...) aktualisiert das belegt-Feld für eine Person anhand der
UID."""

def update_belegt(uid: str, belegt: int, path: Optional[str] = None) -> int:
    # Datenbank wird als Variable geöffnet

```

```
with sqlite3.connect(DB_PATH) as conn:
    cur = conn.cursor() # für SQL Befehle
    try:
        # belegt-Feld aktualisieren
        cur.execute('UPDATE personen SET belegt = ? WHERE uid = ?',
(belegt, uid))
        print(f"UID {uid}: belegt wurde auf {belegt} gesetzt.")
        return cur.rowcount
    except sqlite3.Error as e:
        print(f"Fehler beim Aktualisieren von belegt: {e}")
        return 0

"""check(...) überprüft, ob ein Parkplatz belegt ist anhand der
Parkplatznummer."""

def check(parkplatznummer: str) -> bool:
    # Datenbank wird als Variable geöffnet
    with sqlite3.connect(DB_PATH) as conn:
        cur = conn.cursor() # für SQL Befehle
        cur.execute('SELECT belegt FROM personen WHERE parkplatznummer = ?',
(parkplatznummer,)) # Belegt-Status abfragen
        result = cur.fetchone()
        # Ergebnis zurückgeben
        if result:
            return result[0] == 1
        return False
```

A.2 Datei: gui.py

Beschreibung: Implementierung des grafischen Benutzerinterface (GUI) zur Visualisierung des Tabelleninhalts der Datenbank und ermöglicht die Manipulation der Tabelle.

```
import tkinter as tk
from tkinter import ttk, messagebox
import database
# Projektlogik, Methodenplanung und GUI-/Datenbankstruktur stammen vollständig
vom Autor;
# GitHub Copilot / ChatGPT wurde nur teils für Python-Syntax und
Bibliotheksdetails genutzt.

"""gui.py
GUI für Parkplatzverwaltungssystem
@author: Vincent Gentz
Matrikelnummer: 331843
Datum: 22.10.2025"""

# Datenbankpfad aus database.py importieren, falls nicht vorhanden, wird
erstellt
DB_PATH = database.DB_PATH if hasattr(database, 'DB_PATH') else
'app_database.db'

class ParkGUI(tk.Tk):
    # Initialisierung des Grundfensters der GUI
    def __init__(self):
        super().__init__()
        self.title('Personen - Parkverwaltung')
        self.geometry('1000x420')
        self.create_widgets()
        self.load_data()

    # Erstellung der Widgets
    def create_widgets(self):
        cols = ('UID', 'Vorname', 'Nachname', 'Parkplatznummer', 'Belegt',
'Eingecheckt')
        self.tree = ttk.Treeview(self, columns=cols, show='headings',
height=20)

        # Spaltenbreiten definieren
        self.tree.column('UID', width=100)
        self.tree.column('Vorname', width=120)
```



```

self.tree.column('Nachname', width=120)
self.tree.column('Parkplatznummer', width=120)
self.tree.column('Belegt', width=80)
self.tree.column('Eingecheckt', width=100)

# Spaltenüberschriften definieren
for col in cols:
    self.tree.heading(col, text=col)

# Baumansicht und Scrollbar
self.tree.pack(side='left', fill='both', expand=True, padx=(8,0),
pady=8)

    scrollbar = ttk.Scrollbar(self, orient='vertical',
command=self.tree.yview)
    self.tree.configure(yscrollcommand=scrollbar.set)
    scrollbar.pack(side='left', fill='y', pady=8)

# Kontextmenü für Löschen
self.menu = tk.Menu(self, tearoff=0)
self.menu.add_command(label="Löschen", command=self.delete_selected)
self.tree.bind("<Button-3>", self.show_context_menu)

# Rechte Seite mit Buttons
right = ttk.Frame(self)
right.pack(side='right', fill='y', padx=8, pady=8)

    ttk.Button(right, text="Person hinzufügen",
command=self.Dialogfenster_hinzufuegen, width=20).pack(pady=5)
    ttk.Button(right, text="Aktualisieren", command=self.load_data,
width=20).pack(pady=5)
    ttk.Button(right, text="Beenden", command=self.quit,
width=20).pack(pady=5)

# Kontextmenü anzeigen
def show_context_menu(self, event):
    item = self.tree.identify_row(event.y) # Zeile unter Mauszeiger finden
    if item:
        self.tree.selection_set(item) # Zeile auswählen
        self.menu.post(event.x_root, event.y_root) # Menü anzeigen

# Ausgewählte Person löschen
def delete_selected(self):
    selected = self.tree.selection() # Ausgewählte Zeile holen

    if not selected:

```

```

        messagebox.showwarning("Keine Auswahl", "Bitte wähle eine Person
aus.")

        return

    item = selected[0] #
    values = self.tree.item(item, 'values')
    uid = values[0]

    confirm = messagebox.askyesno("Löschen bestätigen",
                                  f"Person mit UID {uid} wirklich
löschen?")
    if confirm:
        database.delete_person(uid)
        self.load_data()

# Daten aus Datenbank laden und anzeigen
def load_data(self):
    # Alle Einträge löschen
    for item in self.tree.get_children():
        self.tree.delete(item)

    # Neue Einträge laden
    personen = database.show_all_personen()
    for p in personen:
        self.tree.insert('', 'end', values=(
            p['uid'],
            p['vorname'],
            p['nachname'],
            p['parkplatznummer'],
            'Ja' if p['belegt'] else 'Nein',
            'Ja' if p['eingecheckt'] else 'Nein'
        ))

# Dialog zum Hinzufügen einer neuen Person öffnen
def Dialogfenster_hinzufuegen(self):

    dlg = tk.Toplevel(self)
    dlg.title("Person hinzufügen")
    dlg.geometry("350x220")
    #self.center_window(dlg)

    # Eingabefeld für UID, Vorname, Nachname, Parkplatznummer
    ttk.Label(dlg, text="UID:").grid(row=0, column=0, padx=10, pady=5,
sticky='w')
    uid_entry = ttk.Entry(dlg, width=25)
    uid_entry.grid(row=0, column=1, padx=10, pady=5)

```

```

        ttk.Label(dlg, text="Vorname:").grid(row=1, column=0, padx=10, pady=5,
sticky='w')
        vorname_entry = ttk.Entry(dlg, width=25)
        vorname_entry.grid(row=1, column=1, padx=10, pady=5)

        ttk.Label(dlg, text="Nachname:").grid(row=2, column=0, padx=10,
pady=5, sticky='w')
        nachname_entry = ttk.Entry(dlg, width=25)
        nachname_entry.grid(row=2, column=1, padx=10, pady=5)

        ttk.Label(dlg, text="Parkplatznummer:").grid(row=3, column=0, padx=10,
pady=5, sticky='w')
        parkplatz_entry = ttk.Entry(dlg, width=25)
        parkplatz_entry.grid(row=3, column=1, padx=10, pady=5)

        # Hinzufügen-Button mit Hilfe von add_person_gui
        ttk.Button(dlg, text="Hinzufügen",
                    command=lambda: self.add_person_gui(dlg, uid_entry.get(),
vorname_entry.get(),
                                                            nachname_entry.get(),
parkplatz_entry.get())
                    ).grid(row=4, column=0, columnspan=2, pady=10)

        # Kontrolle der Eingaben und Hinzufügen der Person zur Datenbank mit
add_person
        def add_person_gui(self, dlg, uid: str, vorname: str, nachname: str,
parkplatz: str):
            uid = uid.upper().replace(" ", "")

            if not uid or not vorname or not nachname or not parkplatz:
                messagebox.showwarning("Eingabefehler", "Bitte alle Felder
ausfüllen!")
                return

            try:
                database.add_person(uid=uid, vorname=vorname, nachname=nachname,
parkplatznummer=parkplatz) # Fügt Person hinzu
                messagebox.showinfo("Erfolg", f"Person {vorname} {nachname} mit
UID {uid} hinzugefügt!")
                dlg.destroy() # Zwischenspeicher schließen
                self.load_data()
            except Exception as e:
                messagebox.showerror("Fehler", f"Fehler beim Hinzufügen: {e}")

```

```
# Zentriert ein Fenster auf dem Bildschirm
def fokus_window(self, win):
    win.update_idletasks() # Aktualisiert Fensterinformationen
    width = win.winfo_width()
    height = win.winfo_height()
    x = (win.winfo_screenwidth() // 2) - (width // 2)
    y = (win.winfo_screenheight() // 2) - (height // 2)
    win.geometry(f'{width}x{height}+{x}+{y}')

if __name__ == '__main__':
    app = ParkGUI()
    app.mainloop()
```

A.3 Datei: serial_listener.py

Beschreibung: Ist die Schnittstelle zwischen Arduino und Datenbank. Der Listener liest die Ausgabe des Arduinos und greift auf die Datenbankfunktionen zurück.

```
import serial
import time
from database import update_eingecheckht
# Projektlogik, Methodenplanung und GUI-/Datenbankstruktur stammen vollständig
vom Autor;
# GitHub Copilot / ChatGPT wurde nur teils für Python-Syntax und
Bibliotheksdetails genutzt.

"""
serial_listener.py
Skript hört die Schnittstelle zum Arduino ab und aktualisiert die Datenbank.
@author: Vincent Gentz
Matrikelnummer:331843
Datum: 22.10.2025"""
# Konfiguration
PORT = '/dev/ttyACM0'
BAUD = 9600

"""listen() wartet auf UID-Nachrichten vom Arduino und aktualisiert die
Datenbank."""

def listen():
    # versucht Verbindung aufzubauen
    try:
        print(f"Versuche Verbindung zu {PORT} mit {BAUD} Baud...")
        with serial.Serial(PORT, BAUD, timeout=1) as ser:
            print(f"Serial Listener erfolgreich gestartet auf {PORT}")
            print("Warte auf Arduino-Daten")

            time.sleep(2) # Arduino Zeit zum Booten geben

            while True:
                if ser.in_waiting > 0: # Überprüfen, ob Daten im Puffer sind
                    line = ser.readline().decode('utf-8',
errors='ignore').strip() # Zeile lesen und dekodieren

                    if line:
                        print(f"[EMPFANGEN] {line}")

                        # UID-Zeile erkennen: "Karte erkannt! UID: 4C3BE937"
```

```
        if "UID:" in line:
            # UID extrahieren (alles nach "UID: ")
            uid_start = line.find("UID:") + 4
            uid = line[uid_start:].strip().upper()

            # Datenbank aktualisieren
            result = update_eingecheckht(uid)

        time.sleep(0.1) # Kleine Pause zur CPU-Entlastung

except serial.SerialException as e:
    print(f"\nFEHLER beim Öffnen von {PORT}:")
    print(f"    {e}\n")
    print("Überprüfe:")
    print(" Arduino ist angeschlossen (USB-Kabel)")
    print(" COM3 ist der richtige Port (Geräte-Manager prüfen)")
    print(" Arduino IDE Serial Monitor ist GESCHLOSSEN")
    print(" Kein anderes Programm nutzt den Port")

except KeyboardInterrupt:
    print("\n\nSerial Listener beendet (Strg+C)")

except Exception as e:
    print(f"\nUnerwarteter Fehler: {e}")

if __name__ == '__main__':
    print(" RFID Serial Listener")
    listen()
```

A.4 Datei: esp_IP-Kamera_Objekterkennung.py

Beschreibung: Das Skript ruft kontinuierlich Bilder einer Netzwerkkamera ab und identifiziert darauf Objekte mithilfe des YOLOv3-KI-Modells. Erkannte Elemente werden in Echtzeit klassifiziert und visuell mit Begrenzungsrahmen auf dem Bildschirm hervorgehoben.

```
import cv2
import numpy as np
import urllib.request

url = 'http://192.168.178.70/cam-lo.jpg'

cap = cv2.VideoCapture(url)
whT = 320
confThreshold = 0.5
nmsThreshold = 0.3

classesfile = 'coco.names'
with open(classesfile, 'rt') as f:
    classNames = f.read().rstrip('\n').split('\n')

modelConfig = 'yolov3.cfg'
modelWeights = 'yolov3.weights'
net = cv2.dnn.readNetFromDarknet(modelConfig, modelWeights)
net.setPreferableBackend(cv2.dnn.DNN_BACKEND_OPENCV)
net.setPreferableTarget(cv2.dnn.DNN_TARGET_CPU)

def findObject(outputs, im):
    hT, wT, cT = im.shape
    bbox = []
    classIds = []
    confs = []
    found_cat = False
    found_bird = False

    for output in outputs:
        for det in output:
            scores = det[5:]
            classId = np.argmax(scores)
            confidence = scores[classId]
            if confidence > confThreshold:
                w, h = int(det[2]*wT), int(det[3]*hT)
                x, y = int((det[0]*wT) - w/2), int((det[1]*hT) - h/2)
                bbox.append([x, y, w, h])
                classIds.append(classId)
                confs.append(float(confidence))
```

```

indices = cv2.dnn.NMSBoxes(bbox, confs, confThreshold, nmsThreshold)
if len(indices) > 0:
    indices = indices.flatten() # flache Liste, kein i[0] nötig
    for i in indices:
        x, y, w, h = bbox[i]
        if classNames[classIds[i]] == 'bird':
            found_bird = True
        elif classNames[classIds[i]] == 'cat':
            found_cat = True

        cv2.rectangle(im, (x, y), (x + w, y + h), (255, 0, 255), 2)
        cv2.putText(im, f'{classNames[classIds[i]].upper()}
{int(confs[i]*100)}%',
                    (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 0, 255),
2)

while True:
    img_resp = urllib.request.urlopen(url)
    imgnp = np.array(bytearray(img_resp.read()), dtype=np.uint8)
    im = cv2.imdecode(imgnp, -1)

    blob = cv2.dnn.blobFromImage(im, 1/255, (whT, whT), [0, 0, 0], 1, crop=False)
    net.setInput(blob)
    layerNames = net.getLayerNames()
    outputNames = [layerNames[i - 1] for i in
net.getUnconnectedOutLayers().flatten()]
    outputs = net.forward(outputNames)

    findObject(outputs, im)

    cv2.imshow('Image', im)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()

```

Anhang B Arduino-Code

B.1 Main.ino

```
1  #include <SPI.h>
2  #include <MFRC522.h>
3  #include <Adafruit_NeoPixel.h>
4  #include <Wire.h>
5  #include <RTClib.h>
6  #include <LiquidCrystal_I2C.h>
7
8  // RFID
9  #define SS_PIN 10
10 #define RST_PIN 9
11
12 // Ultraschall
13 #define TRIG_PIN 6
14 #define ECHO_PIN 7
15
16 // LED-Leiste
17 #define LED_PIN 8
18 #define NUM_LEDS 8 // 8 LEDs auf der Leiste
19
20 // Objekte
21 MFRC522 mfr522(SS_PIN, RST_PIN);
22 Adafruit_NeoPixel strip(NUM_LEDS, LED_PIN, NEO_GRB + NEO_KHZ800);
23 RTC_DS1307 rtc;
24 LiquidCrystal_I2C lcd(0x27, 16, 2); // LCD mit I2C-Adresse 0x27, Spalten, Zeilen
25
26 // Globale Variablen
27 unsigned long cooldownStart = 0;
28 const unsigned long cooldownTime = 15000; // 15 Sekunden
29 bool cooldownActive = false;
30 bool status = false; //für Anzeige
31 DateTime checkInTime; // Zeitpunkt beim Einchecken
32
33 /*
34 Skript.ino
35 Arduino Skript für Sensoren, LED, RFID, LCD & RTC
36 @author: Vincent Gentz
37 Matrikelnummer:
38 Datum: 22.10.2025
39 */
```

```
41 void setup() {
42     Serial.begin(9600);
43     while (!Serial);
44
45     // RFID Setup
46     SPI.begin();
47     mfrc522.PCD_Init();
48
49     // Ultraschall Setup
50     pinMode(TRIG_PIN, OUTPUT);
51     pinMode(ECHO_PIN, INPUT);
52
53     // LED-Leiste Setup
54     strip.begin();
55     strip.show(); // Alle LEDs aus
56
57     // LCD & RTC Setup
58     lcd.init();
59     lcd.backlight();
60
61
62     rtc.begin();
63
64     if (!rtc.begin()) {
65         Serial.println(F("RTC nicht gefunden"));
66         lcd.print("RTC Fehler!");
67         while (1);
68     }
69
70     rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
71
72     Serial.println(F("RFID + Ultraschall + LED System gestartet"));
73     Serial.println(F("Karte auflegen zum Einchecken"));
74     lcd.clear();
75     lcd.setCursor(0, 0);
76     lcd.print("Parksystem");
77 }
```

```
83 void loop() {
84
85     leddim();
86
87     // Ultraschall-Messung
88     long distance = measureDistance();
89     updateLEDs(distance);
90
91     ///Cooldown runterzählen
92     if (cooldownActive) {
93
94         //millis() verstrichene Zeit seit Start von Controller
95         //timePassed = wie viel Zeit seit Cooldownstart vergangen ist
96         unsigned long timePassed = millis() - cooldownStart;
97         //übrige Zeit
98         unsigned long remaining = cooldownTime - timePassed;
99
100        if (millis() - lastPrint >= 1000) {
101            lastPrint = millis();
102            Serial.print(F("Cooldown aktiv: "));
103            Serial.print(remaining / 1000);
104            Serial.println(F(" Sekunden verbleibend"));
105        }
106
107        if (timePassed >= cooldownTime) {
108            cooldownActive = false;
109            Serial.println(F("Cooldown vorbei - neue Karten können gescannt werden."));
110        }
111        return;
112    }
113
114    else{
115
116        // Überprüfung ob Karte vorhanden ist
117        if (!mfrc522.PICC_IsNewCardPresent()) return;
118        if (!mfrc522.PICC_ReadCardSerial()) return;
```

```
121 String uid = "";
122 //Struct mfrc522.uid gegeben
123 /*
124 |   |   |   size
125 |   |   |   uidByte[]
126 |   |   |   sak (Kartentyp)
127 */
128 for (byte i = 0; i < mfrc522.uid.size; i++) {
129
130     if (mfrc522.uid.uidByte[i] < 0x10) uid += "0";
131     //wandelt uidByte in Hex um, HEX bestimmt Format
132     uid += String(mfrc522.uid.uidByte[i], HEX);
133 }
134 uid.toUpperCase();
135
136 Serial.print(F("Karte erkannt! UID: "));
137
138 // Status umschalten (Ein/Auschecken)
139 status = !status;
140
141 display(status); // LCD-Funktion
142
143 Serial.println(uid);
144
145 cooldownStart = millis();
146 cooldownActive = true;
147 //für Cooldown Ausgabe pro Sekunde
148 lastPrint = 0;
149
150 mfrc522.PICC_HaltA();
151 mfrc522.PCD_StopCrypto1();
152 }
153
154
155 strip.show();
156 }
```

B.2 LCD.ino

```
1  #include <Wire.h>
2  #include <RTClib.h>
3  #include <LiquidCrystal_I2C.h>
4
5  /*
6  von Serhat Subasi
7  Matrikelnummer: 327488
8  */
9
10
11
12 // Zugriff auf globale Variablen
13 extern RTC_DS1307 rtc;
14 extern LiquidCrystal_I2C lcd;
15 extern DateTime checkInTime;
16
17 void display(bool position) {
18     lcd.clear();          //räumt den Bildschirm auf
19
20
21     if (position) {
22         // Einchecken
23         checkInTime = rtc.now();
24         lcd.setCursor(0, 0);    // setzt auf die erste Zeile und erste Spalte auf dem Display
25         lcd.print("Es wurde");  // gibt den Text auf dem LCD Modul aus
26         lcd.setCursor(0, 1);    // setzt auf die zweite Zeile
27         lcd.print("Eingecheckt!");
28         Serial.println(F("Es wurde eingecheckt"));
29
30         delay(5000);
31         lcd.clear();
32     }
33 }
```

```
33 else {
34     // Auschecken
35     DateTime checkOutTime = rtc.now(); // holt die aktuelle Zeit
36     TimeSpan parkdauer = checkOutTime - checkInTime; // berechnet die Parkdauer durch die Differenz
37
38     lcd.setCursor(0, 0);
39     lcd.print("Es wurde");
40     lcd.setCursor(0, 1);
41     lcd.print("ausgecheckt");
42
43     delay(3000); // 2 Sekunden Pause, bevor die Parkdauer angezeigt wird
44     lcd.clear();
45
46     lcd.setCursor(0, 0);
47     lcd.print("Parkdauer:");
48     lcd.setCursor(0, 1);
49     lcd.print(parkdauer.hours()); // gibt die Stunde aus
50     lcd.print("h ");
51     lcd.print(parkdauer.minutes()); // gibt die Minute aus
52     lcd.print("m ");
53     lcd.print(parkdauer.seconds()); // gibt die Sekunde aus
54     lcd.print("s");
55
56     Serial.print(F("Es wurde ausgecheckt - Parkdauer: "));
57     Serial.print(parkdauer.hours());
58     Serial.print(" Stunden, ");
59     Serial.print(parkdauer.minutes());
60     Serial.print(" Minuten, ");
61     Serial.print(parkdauer.seconds());
62     Serial.println(" Sekunden");
63
64     delay(5000);
65     lcd.clear();
66 }
67 }
```

B.3 Distance.ino

```
1  #include <SPI.h>
2  #include <MFRC522.h>
3  #include <Adafruit_NeoPixel.h>
4
5  long measureDistance() {
6      digitalWrite(TRIG_PIN, LOW);
7      delayMicroseconds(2);
8      digitalWrite(TRIG_PIN, HIGH);
9      delayMicroseconds(10);
10     digitalWrite(TRIG_PIN, LOW);
11
12     long duration = pulseIn(ECHO_PIN, HIGH);
13     long distance = duration * 0.034 / 2;
14
15     return distance;
16 }
```

B.4 Leddim.ino

```
1  #include <Adafruit_NeoPixel.h>
2  #include <RTClib.h>
3
4  /*
5  von Serhat Subasi
6  Matrikelnummer: 327488
7  */
8
9
10
11 // Zugriff auf globale Variablen
12 extern RTC_DS1307 rtc;
13 extern Adafruit_NeoPixel strip;
14
15
16 unsigned long ausgabe = 0;
17
18
19 void leddim() {
20     DateTime now = rtc.now(); // aktuelle Zeit abrufen
21     int stunde = now.hour();
22
23
24     // Debug-Ausgabe
25     /*
26     if (millis() - ausgabe >= 1000) {
27         Serial.print(F("Aktuelle Uhrzeit: "));
28         Serial.print(stunde);
29         Serial.print(F(":"));
30         Serial.println(now.minute());
31         ausgabe = millis();
32     }
33     */
34
35     // Dimmung ab 20 Uhr
36
37     if (stunde >= 20 || stunde < 8) { //prüft ob es zwischen 20 und 8 Uhr ist
38         strip.setBrightness(40); // Helligkeit reduzieren (0-255)
39     } else {
40         strip.setBrightness(150); // Helligkeit erhöhen
41     }
42
43 }
```


B.5 Updateled.ino

```
1  #include <SPI.h>
2  #include <MFRC522.h>
3  #include <Adafruit_NeoPixel.h>
4
5
6  void updateLEDs(long distance) {
7      uint32_t color;
8
9      if (distance > 100) {          // > 1m = GRÜN
10         color = strip.Color(0, 255, 0);
11     }
12     else if (distance >= 50) {      // 50cm - 1m = ORANGE
13         color = strip.Color(255, 165, 0);
14     }
15     else {                          // < 50cm = ROT
16         color = strip.Color(255, 0, 0);
17     }
18
19     // Alle 8 LEDs auf die Farbe setzen
20     for(int i = 0; i < 8; i++) {
21         strip.setPixelColor(i, color);
22     }
23     strip.show();
24
25     // Debug-Ausgabe nur alle 2 Sekunden
26     static unsigned long lastDistancePrint = 0;
27     if (millis() - lastDistancePrint >= 2000) {
28         lastDistancePrint = millis();
29         Serial.print(F("Entfernung: "));
30         Serial.print(distance);
31         Serial.print(F(" cm - "));
32         if (distance > 100) Serial.println(F("GRÜN"));
33         else if (distance >= 50) Serial.println(F("ORANGE"));
34         else Serial.println(F("ROT"));
35     }
36
37     //delay(100); // Kleine Pause zwischen Messungen
38 }
```

B.6 Datei: WIFICam.ino

Beschreibung: Dieser Code konfiguriert den Mikrokontroller als Webserver, der auf Anfrage aktuelle JPEG-Fotos aufnimmt und über das WLAN versendet. Er verwaltet die Kameraeinstellungen und stellt verschiedene Auflösungen über spezifische URL-Pfade bereit.

```
#include <WebServer.h>
#include <WiFi.h>
#include <esp32cam.h>

const char* WIFI_SSID = "";
const char* WIFI_PASS = "";

WebServer server(80);

static auto loRes = esp32cam::Resolution::find(320, 240);
static auto midRes = esp32cam::Resolution::find(350, 530);
static auto hiRes = esp32cam::Resolution::find(800, 600);
void serveJpg()
{
    auto frame = esp32cam::capture();
    if (frame == nullptr) {
        Serial.println("CAPTURE FAIL");
        server.send(503, "", "");
        return;
    }
    Serial.printf("CAPTURE OK %dx%d %db\n", frame->getWidth(), frame->getHeight(),
        static_cast<int>(frame->size()));

    server.setContentLength(frame->size());
    server.send(200, "image/jpeg");
    WiFiClient client = server.client();
    frame->writeTo(client);
}

void handleJpgLo()
{
    if (!esp32cam::Camera.changeResolution(loRes)) {
        Serial.println("SET-LO-RES FAIL");
    }
    serveJpg();
}

void handleJpgHi()
{
    if (!esp32cam::Camera.changeResolution(hiRes)) {
        Serial.println("SET-HI-RES FAIL");
    }
}
```

```
    }
    serveJpg();
}

void handleJpgMid()
{
    if (!esp32cam::Camera.changeResolution(midRes)) {
        Serial.println("SET-MID-RES FAIL");
    }
    serveJpg();
}

void setup(){
    Serial.begin(115200);
    Serial.println();
    {
        using namespace esp32cam;
        Config cfg;
        cfg.setPins(pins::AiThinker);
        cfg.setResolution(hiRes);
        cfg.setBufferCount(2);
        cfg.setJpeg(80);

        bool ok = Camera.begin(cfg);
        Serial.println(ok ? "CAMERA OK" : "CAMERA FAIL");
    }
    WiFi.persistent(false);
    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASS);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
    }
    Serial.print("http://");
    Serial.println(WiFi.localIP());
    Serial.println(" /cam-lo.jpg");
    Serial.println(" /cam-hi.jpg");
    Serial.println(" /cam-mid.jpg");

    server.on("/cam-lo.jpg", handleJpgLo);
    server.on("/cam-hi.jpg", handleJpgHi);
    server.on("/cam-mid.jpg", handleJpgMid);

    server.begin();
}

void loop()
{
    server.handleClient();
}
```

Formelzeichenverzeichnis

t	Zeit
d	Distanz
c_{Schall}	Schallgeschwindigkeit
μs	Mikrosekunde
ms	Millisekunden
